

Subject : Data Structures & Algorithms

Lecture : 05

Sorting Algorithms

Some Definitions

- **Sorting**

- Is a process that organizes a collection of data into either ascending or descending order

- **Internal Sort**

- The data to be sorted is all stored in the computer's main memory.

- **External Sort**

- Some of the data to be sorted might be stored in some external, slower, device.

- **In Place Sort**

- The amount of extra space required to sort the data is constant with the input size.

Sorting

- We will analyze only **internal sorting** algorithms.
- Sorting also has indirect uses. An initial sort of the data can significantly enhance the performance of an algorithm.
- Majority of programming projects use a sort somewhere, and in many cases, the sorting cost determines the running time.
- It is estimated that 25~50% of all computing power is used for sorting activities.

Sorting Applications

- **Common problem:** sort a list of values, starting from lowest to highest.
- To prepare a list of student ID, names, and scores in a table (sorted by ID or name) for easy checking.
- To prepare a list of scores before letter grade assignment.
- To produce a list of horses after a race (sorted by the finishing times) for payoff calculation.

Sorting Applications...

- To prepare an originally unsorted array for ordered binary searching.
- List of exam scores
- Words of dictionary in alphabetical order
- **Generally**, we are given a list of records that have keys. These keys are used to define an ordering of the items in the list.

C++ Implementation of Sorting

- Use C++ templates to implement a generic sorting function.
- This would allow use of the same function to sort items of any class.
- However, class to be sorted must provide the following overloaded operators:
 - **Assignment:** =
 - **Ordering:** >, <, ==
- In this lecture, we'll talk about sorting integers; however, the algorithms are general and can be applied to any class as described above.

Sorting Algorithms

- There are many sorting algorithms, such as:
 - Selection Sort
 - Insertion Sort
 - Bubble Sort
 - Merge Sort
 - Quick Sort
- The first three are the foundations for faster and more efficient algorithms.

Selection Sort

- Selection sort is a simple sorting algorithm. It works by first finding the smallest element using a linear search and swapping it into the first position in the list,
- Then finding the second smallest element by scanning the remaining elements, and so on.
- Selection sort is unique compared to almost any other algorithm in that its running time is not affected by the prior ordering of the list, it performs the same number of operations because of its simple structure.
- A list of n elements requires $n-1$ passes to completely rearrange the data.

Unsorted

23	78	45	8	32	56
----	----	----	---	----	----

Original List

8	78	45	23	32	56
---	----	----	----	----	----

After pass 1

8	23	45	78	32	56
---	----	----	----	----	----

After pass 2

8	23	32	78	45	56
---	----	----	----	----	----

After pass 3

8	23	32	45	78	56
---	----	----	----	----	----

After pass 4

8	23	32	45	56	78
---	----	----	----	----	----

After pass 5

Selection Sort

```
template <class Item>
void selectionSort( Item a[], int n) {
    for (int i = 0; i < n-1; i++) {
        int min = i;
        for (int j = i+1; j < n; j++)
            if (a[j] < a[min])
                min = j;
        swap(a[i], a[min]);
    }
}
```

```
template < class Object>
void swap( Object &lhs, Object &rhs )
{
    Object tmp = lhs;
    lhs = rhs;
    rhs = tmp;
}
```

Insertion Sort

- Insertion sort is a simple sorting algorithm that is appropriate for small inputs.
 - Most common sorting technique used by card players.
- Step 1: The second element of an array is compared with the elements that appears before it (only first element in this case). If the second element is smaller than first element, second element is inserted in the position of first element. After first step, first two elements of an array will be sorted.

Insertion Sort

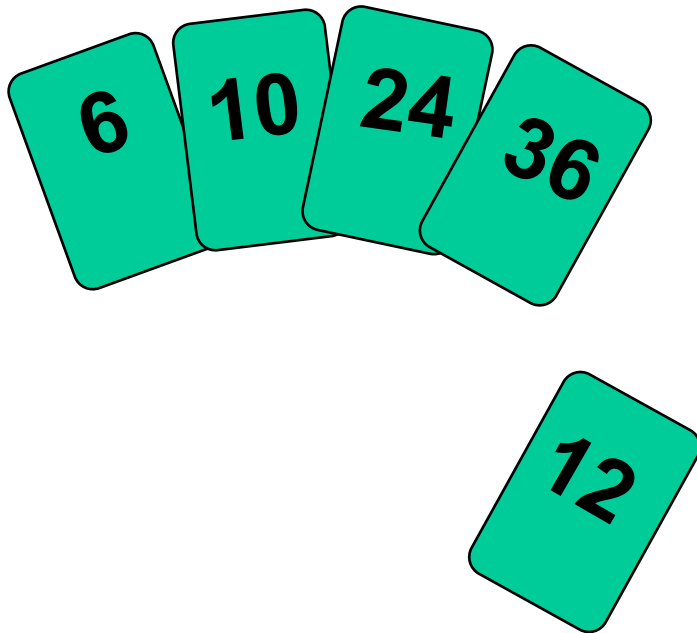
- **Step 2:** The third element of an array is compared with the elements that appear before it (first and second element). If third element is smaller than first element, it is inserted in the position of first element. If third element is larger than first element but, smaller than second element, it is inserted in the position of second element. If third element is larger than both the elements, it is kept in the position as it is. After second step, first three elements of an array will be sorted.
- **Step 3:** Similarly, the fourth element of an array is compared with the elements that appear before it (first, second and third element) and the same procedure is applied and that element is inserted in the proper position.
- If there are n elements to be sorted. Then, this procedure is repeated $n-1$ times to get sorted list of array.

Insertion Sort

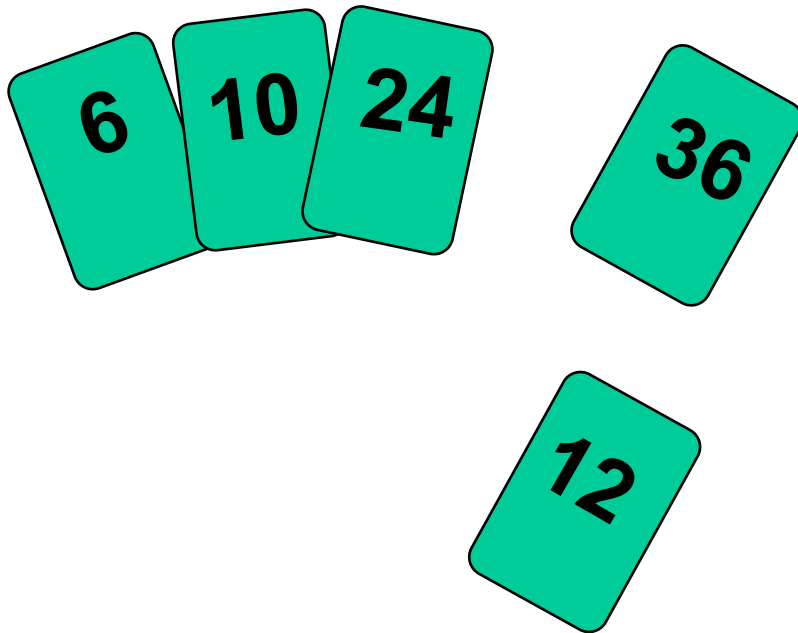
- Sorting a hand of playing cards
 - Start with an empty left hand and the cards facing down on the table.
 - Remove one card at a time from the table, and insert it into the correct position in the left hand
 - compare it with each of the cards already in the hand, from right to left
 - The cards held in the left hand are sorted
 - these cards were originally the top cards of the pile on the table

Insertion Sort

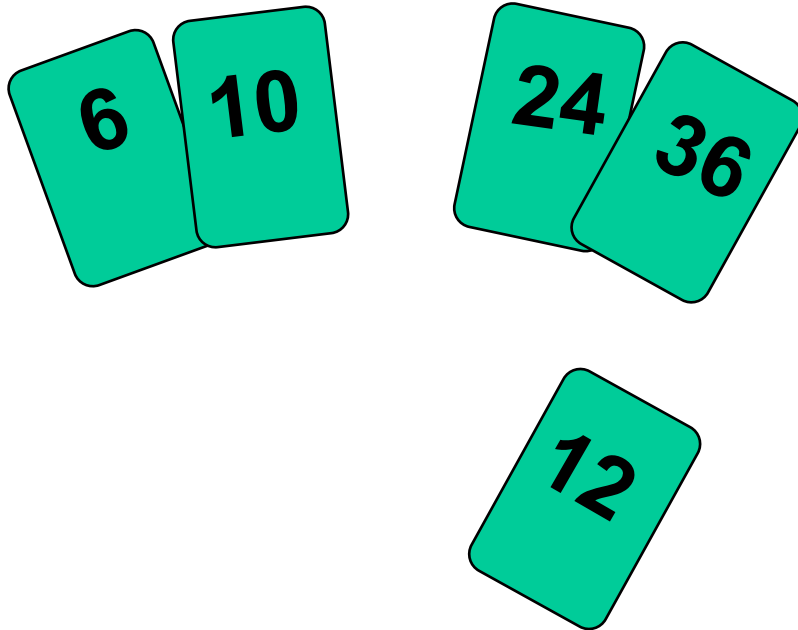
To insert 12, we need to make room for it by moving first 36 and then 24.



Insertion Sort



Insertion Sort



Sorted

Unsorted

23	78	45	8	32	56
----	----	----	---	----	----

Original List

23	78	45	8	32	56
----	----	----	---	----	----

After pass 1

23	45	78	8	32	56
----	----	----	---	----	----

After pass 2

8	23	45	78	32	56
---	----	----	----	----	----

After pass 3

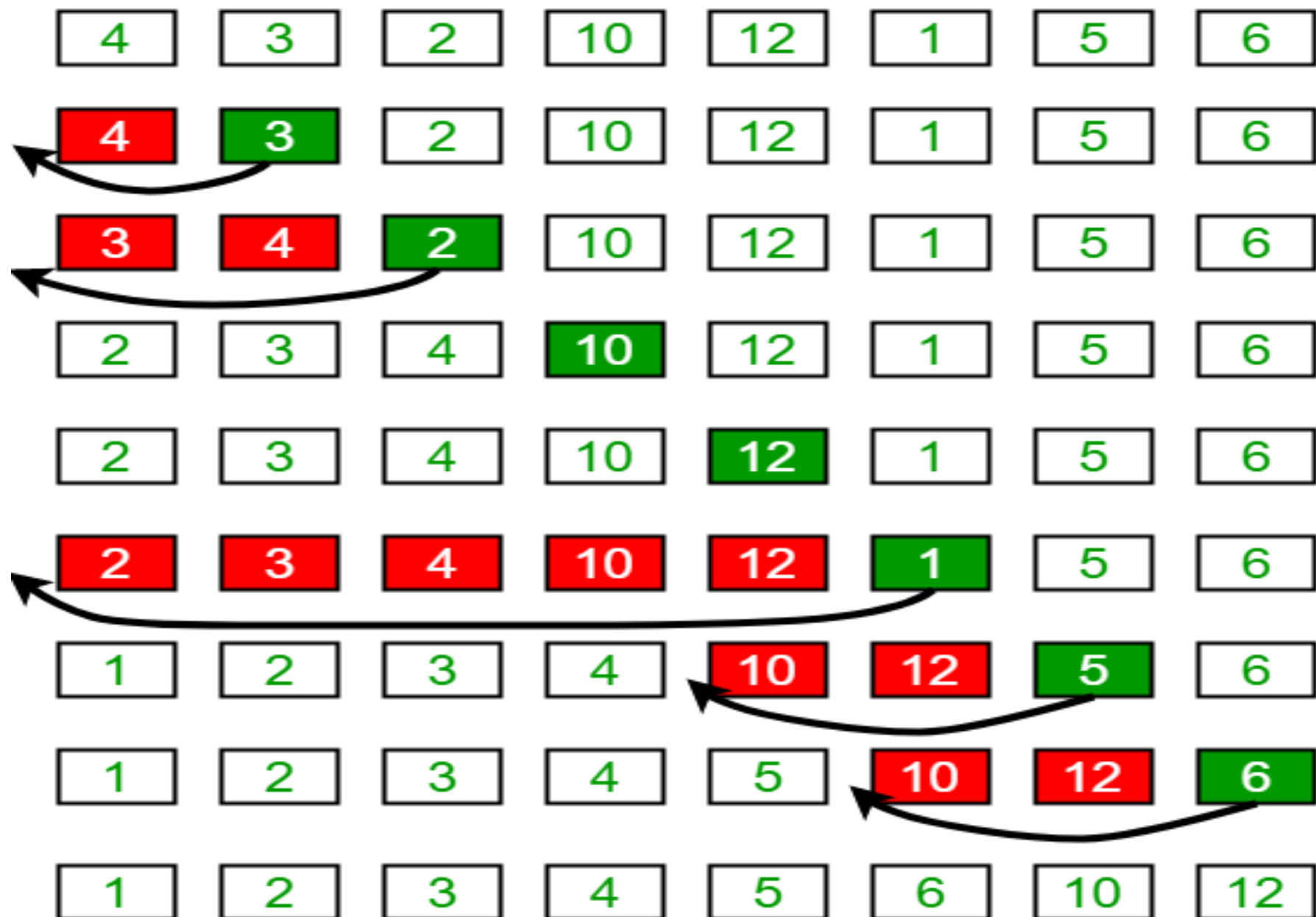
8	23	32	45	78	56
---	----	----	----	----	----

After pass 4

8	23	32	45	56	78
---	----	----	----	----	----

After pass 5

Insertion Sort Execution Example



Insertion Sort Algorithm

```
template <class Item>
void insertionSort(Item arr[], int n)
{
    int i, key, j;
    for (i = 1; i < n; i++)
    {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Bubble Sort

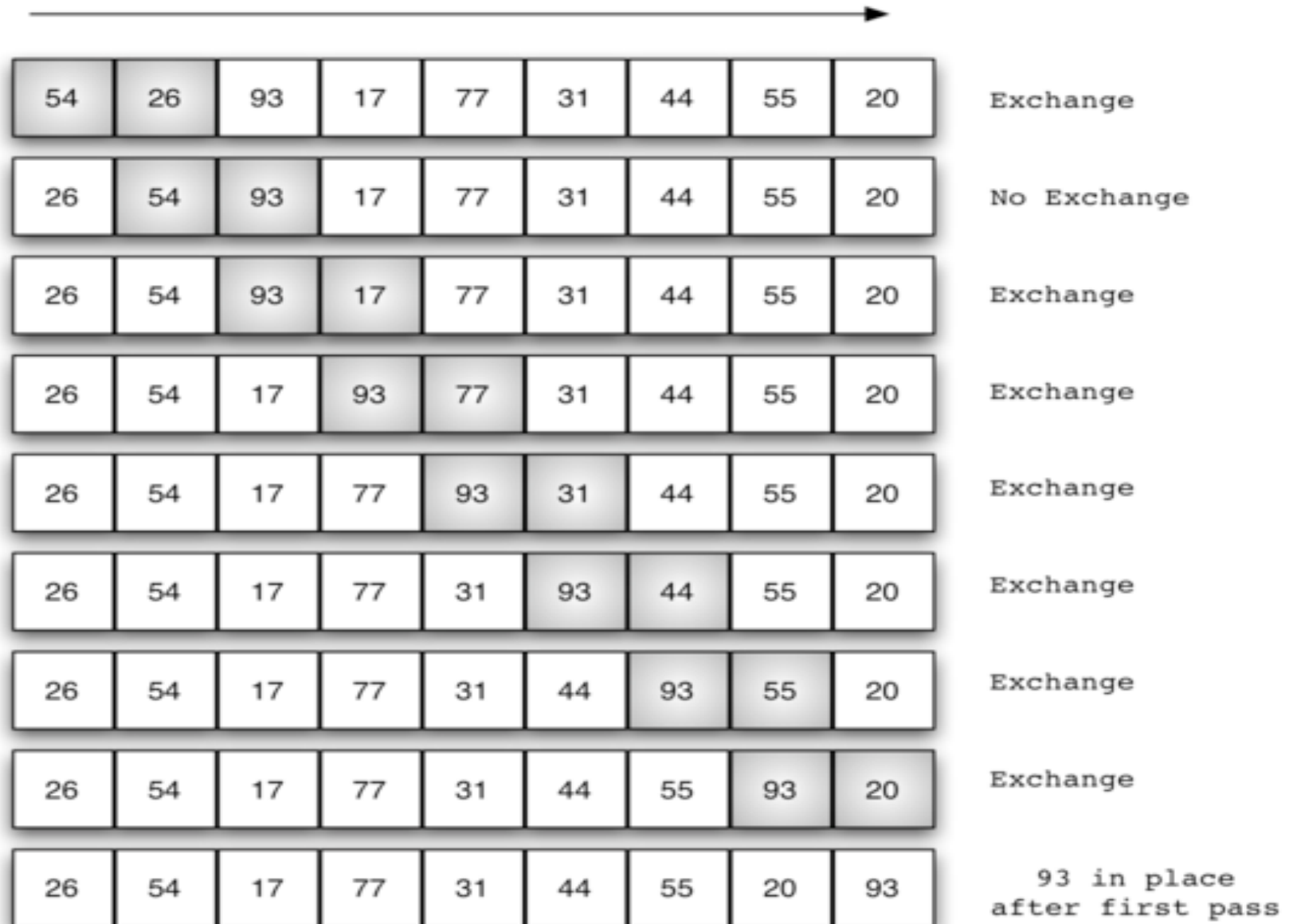
- Bubble sort, sometimes incorrectly referred to as sinking sort, is a simple sorting algorithm that works by repeatedly stepping through the list to be sorted, comparing each pair of adjacent items and swapping them if they are in the wrong order.
- The pass through the list is repeated until no swaps are needed, which indicates that the list is sorted.
- Each time an element moves from the unsorted part to the sorted part one sort pass is completed.

Bubble Sort...

- Given a list of n elements, bubble sort requires up to $n-1$ passes to sort the data.
- Smaller elements "bubble" to the top of the list. Because it only uses comparisons to operate on elements, it is a comparison sort.

Bubble Sort

First pass



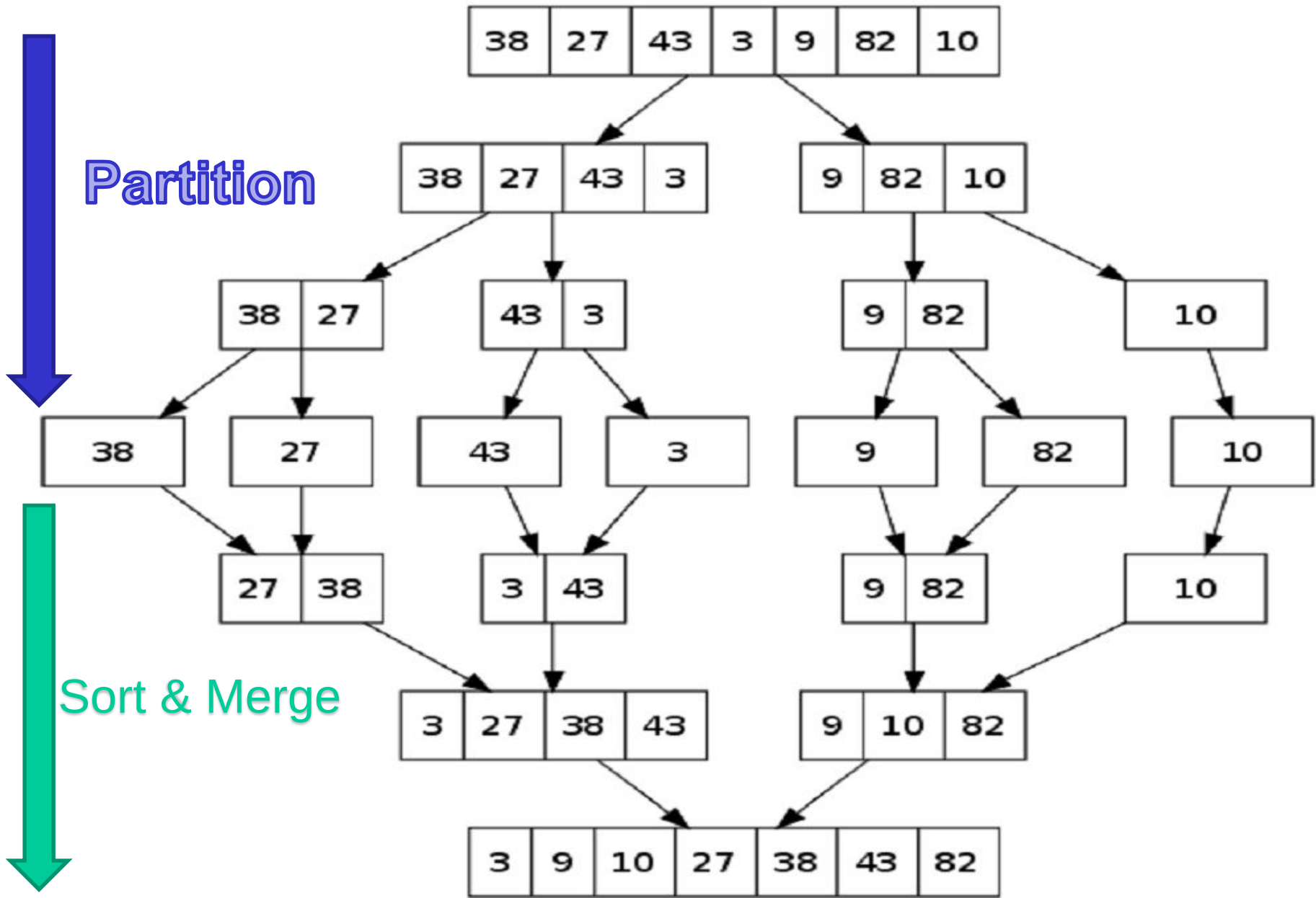
Bubble Sort Algorithm

```
template <class Item>
void bubbleSort(Item a[], int n)
{
    for (int i = 0; i < n; i++) {
        {
            for (int j=0; j < n-1; j++)
            {
                if (a[j] > a[j+1])
                {
                    int temp = a[j+1];
                    a[j+1] = a[j];
                    a[j] = temp;
                }
            }
        }
    }
}
```

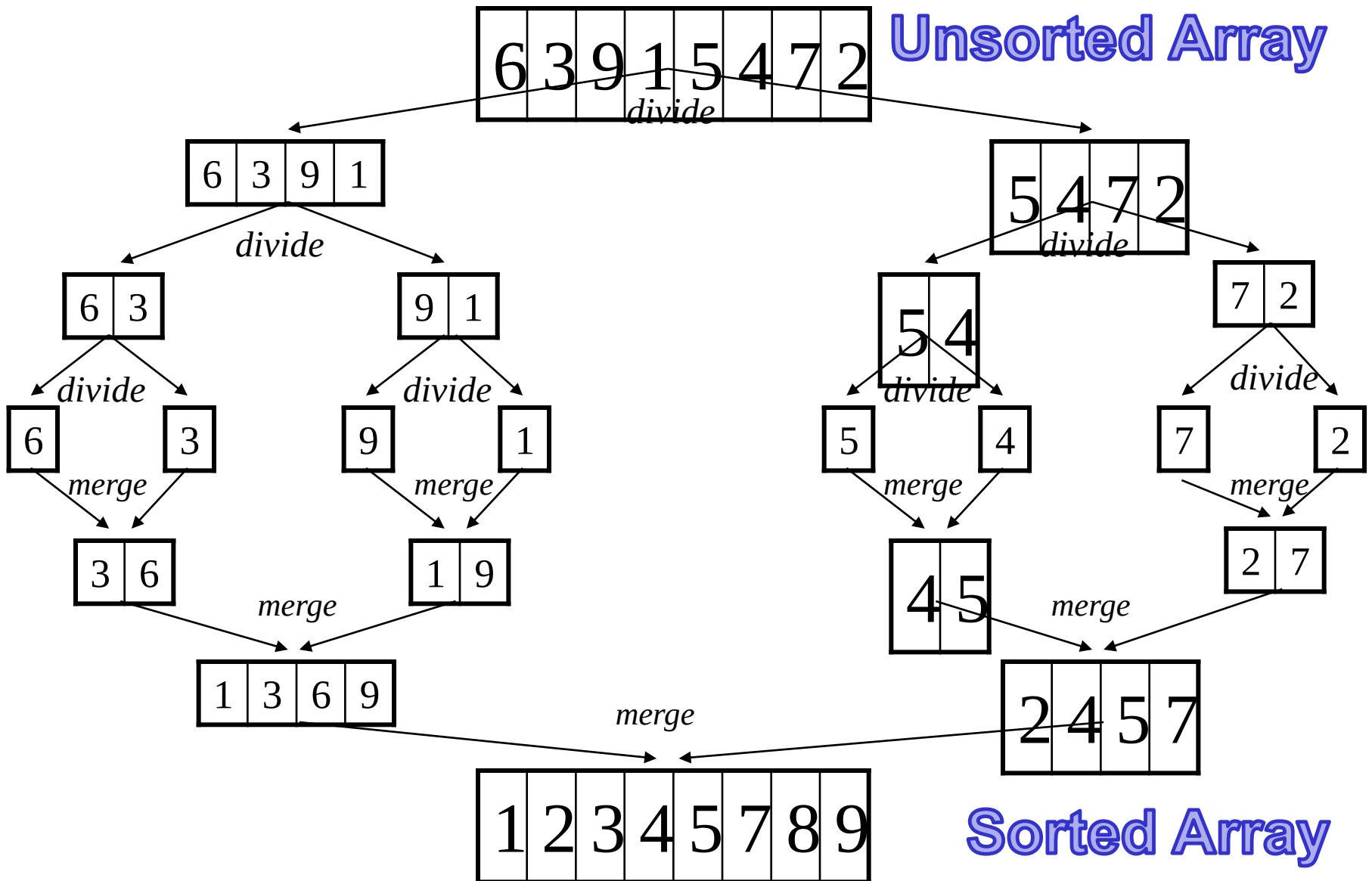

Mergesort

- Merge sort algorithm is one of two important **divide-and-conquer** sorting algorithms (the other one is quicksort).
- It is a recursive algorithm.
 - Divides the list into halves,
 - Sort each half separately, and
 - Then merge the sorted halves into one sorted array.

Mergesort - Example



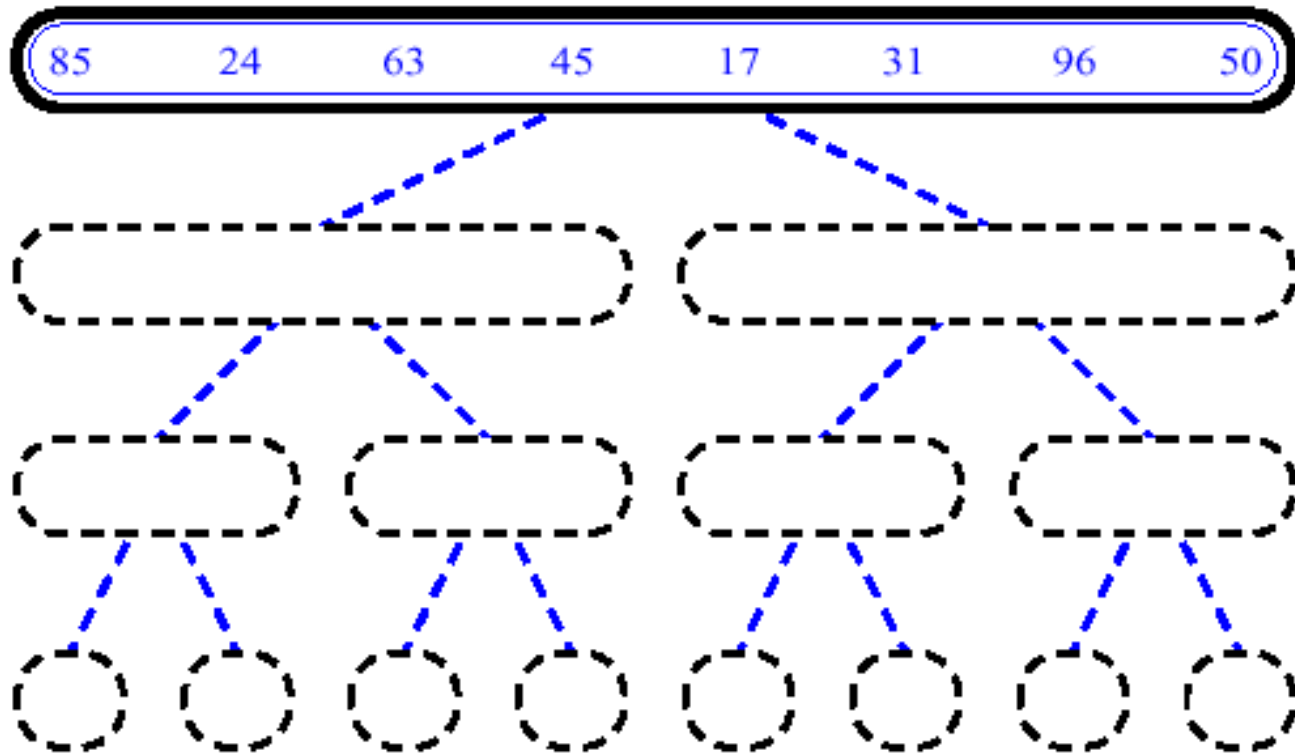
Mergesort - Example



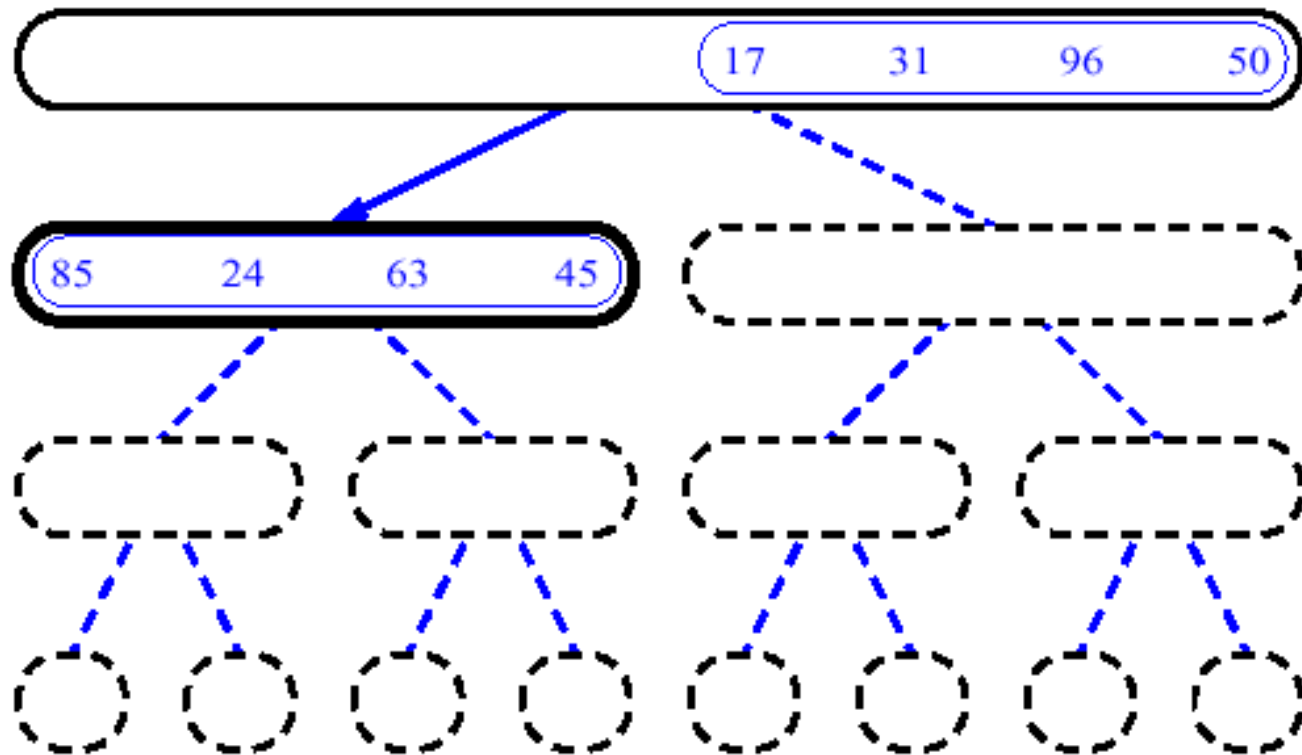
MergeSort (Example) - 1

0	1	2	3	4	5	6	7	8
15	5	24	8	1	3	16	10	20

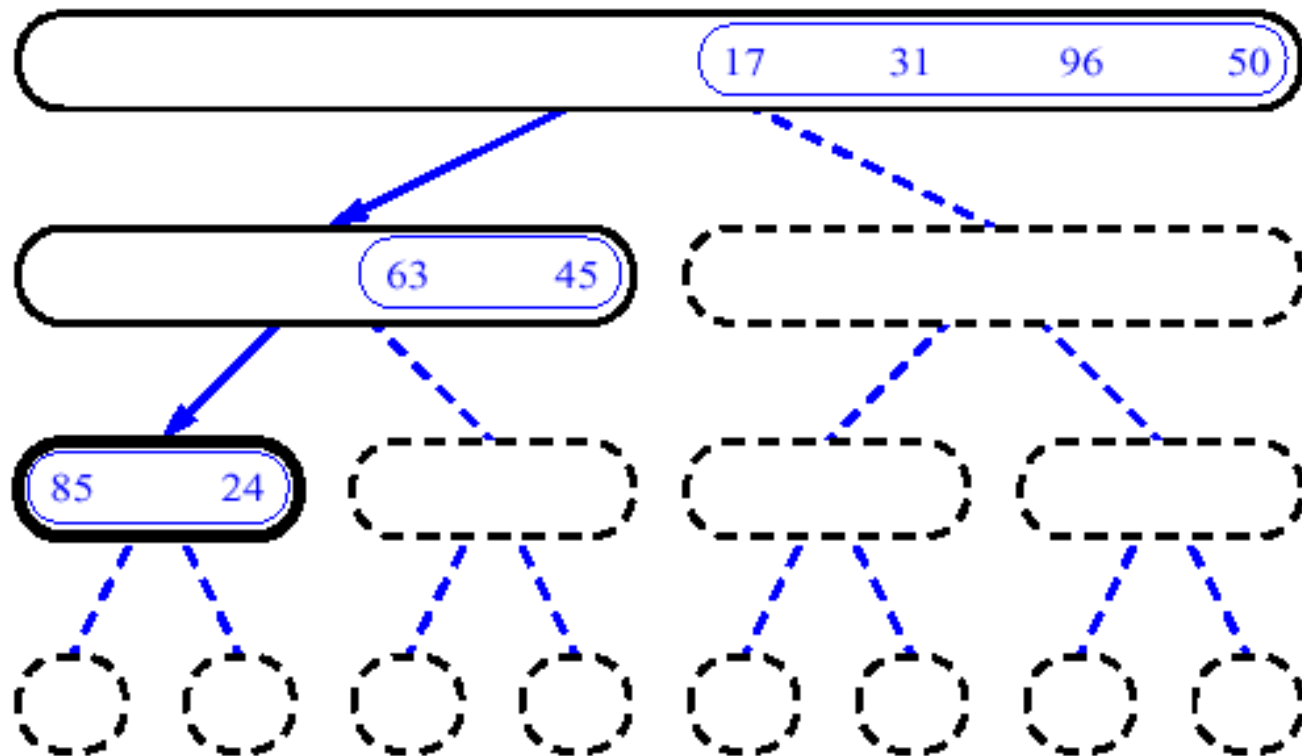
MergeSort (Example) - 1



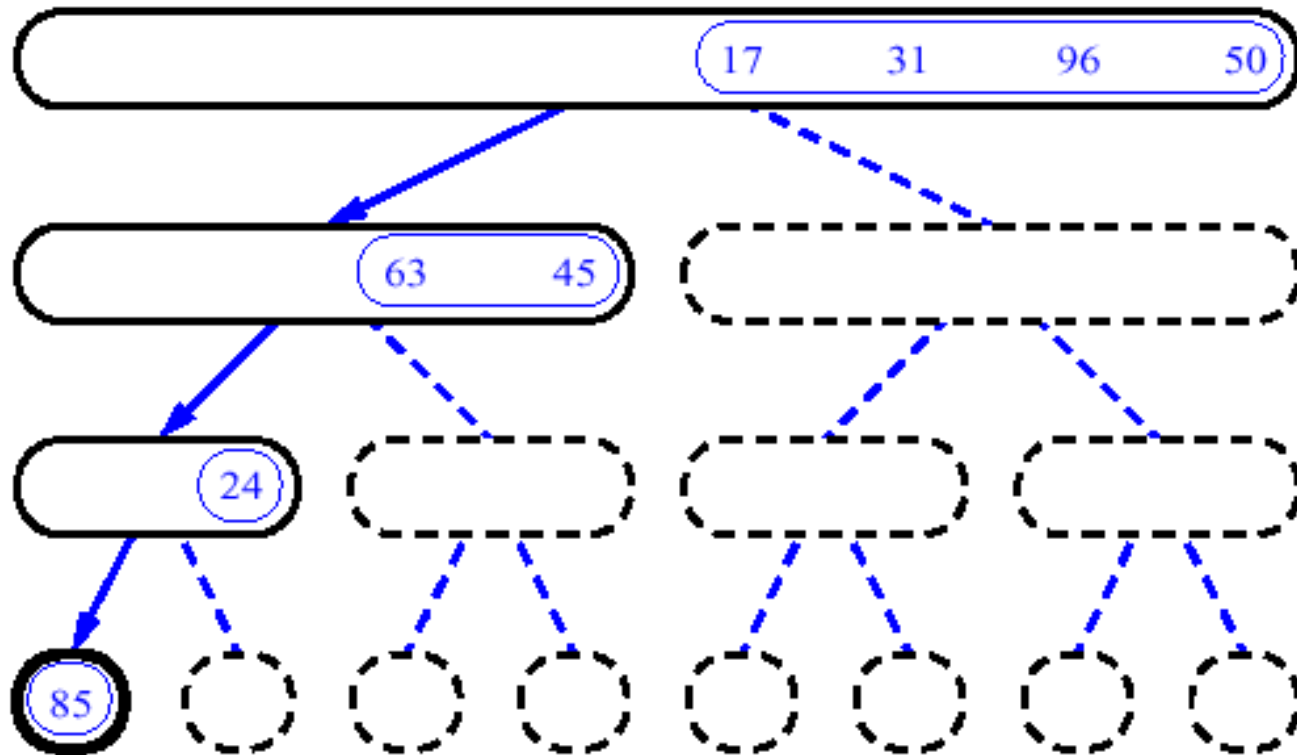
MergeSort (Example) - 2



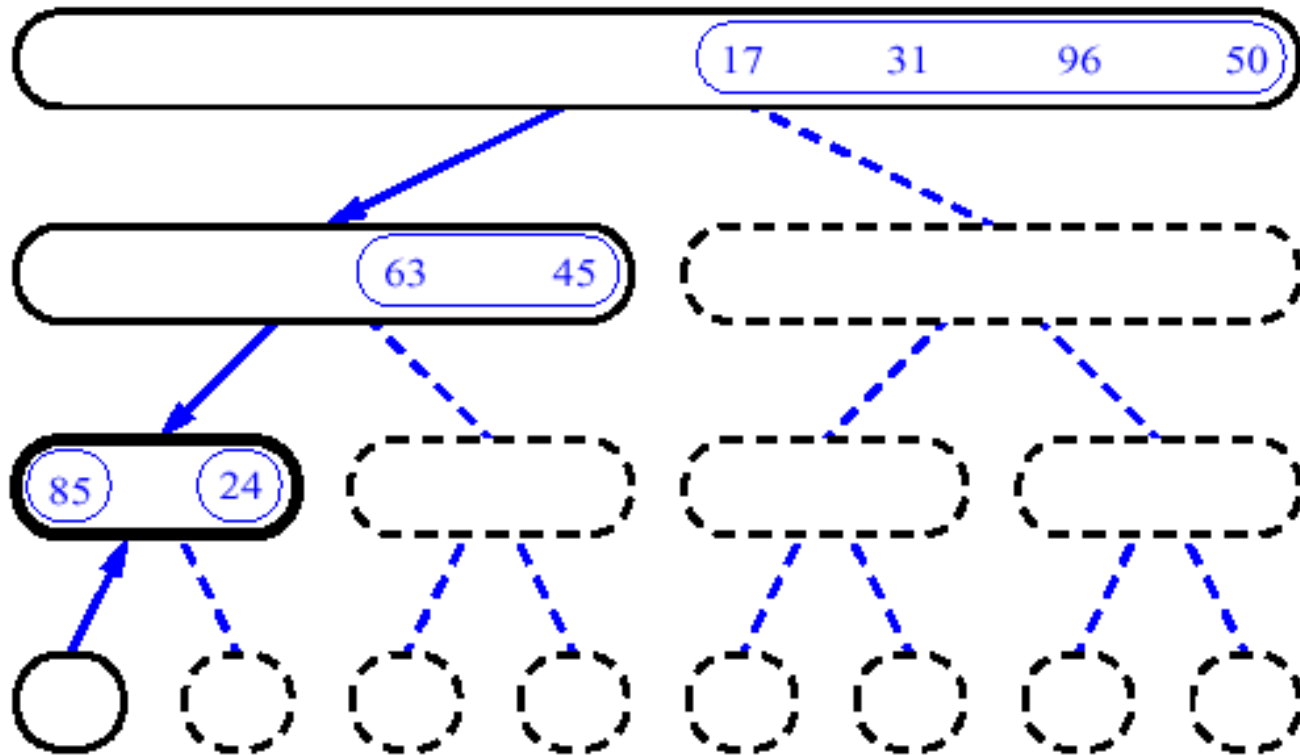
MergeSort (Example) - 3



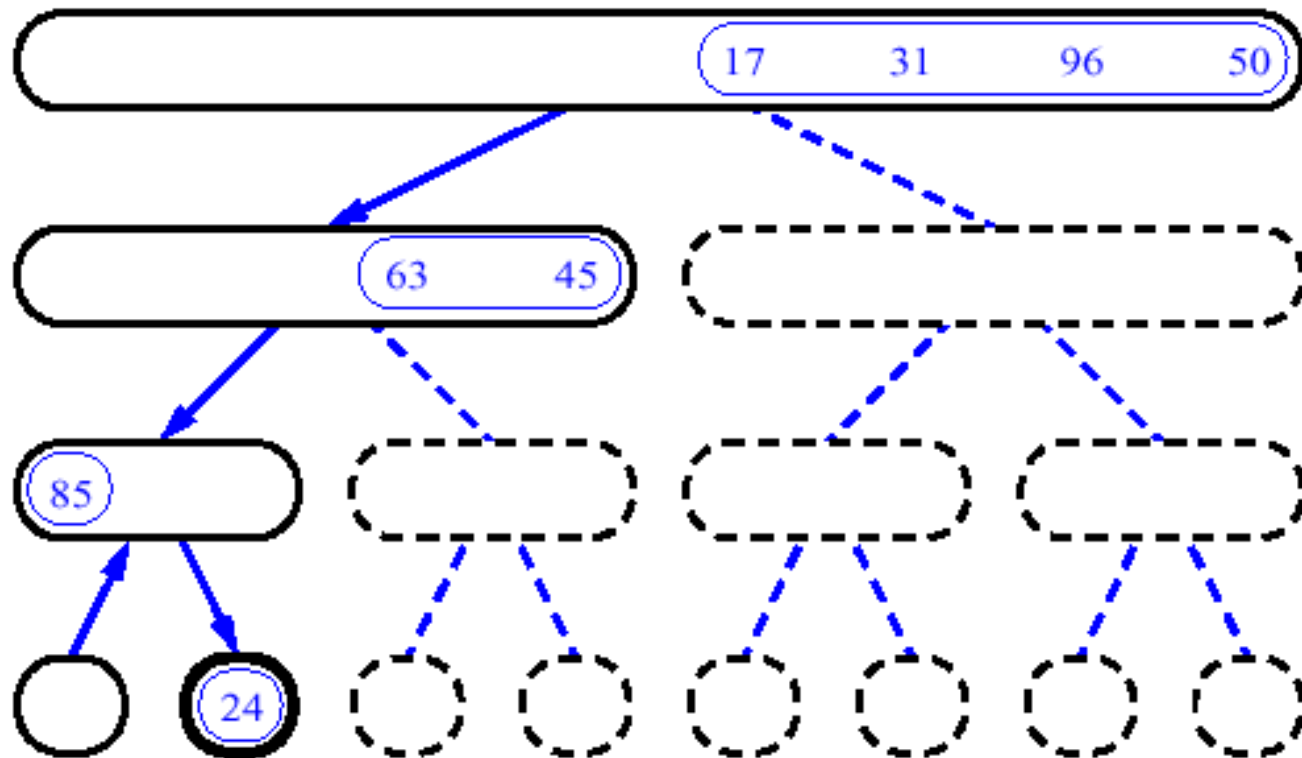
MergeSort (Example) - 4



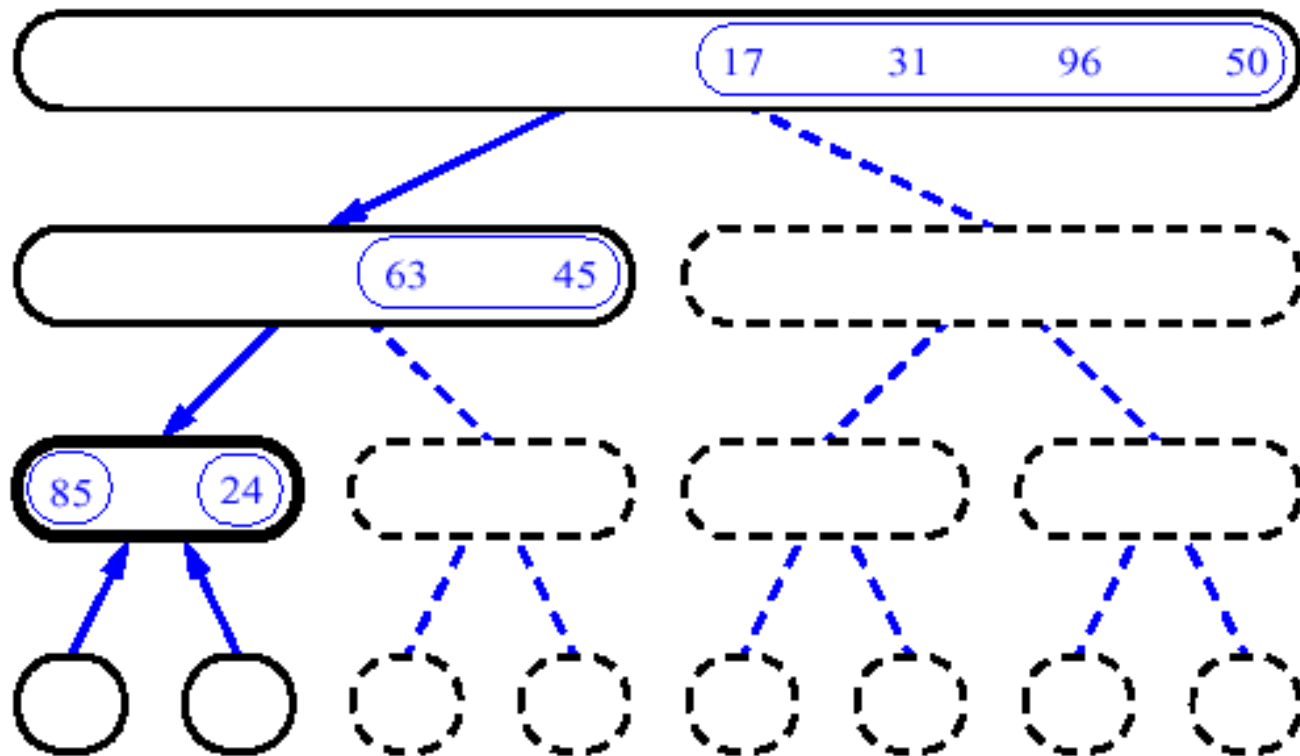
MergeSort (Example) - 5



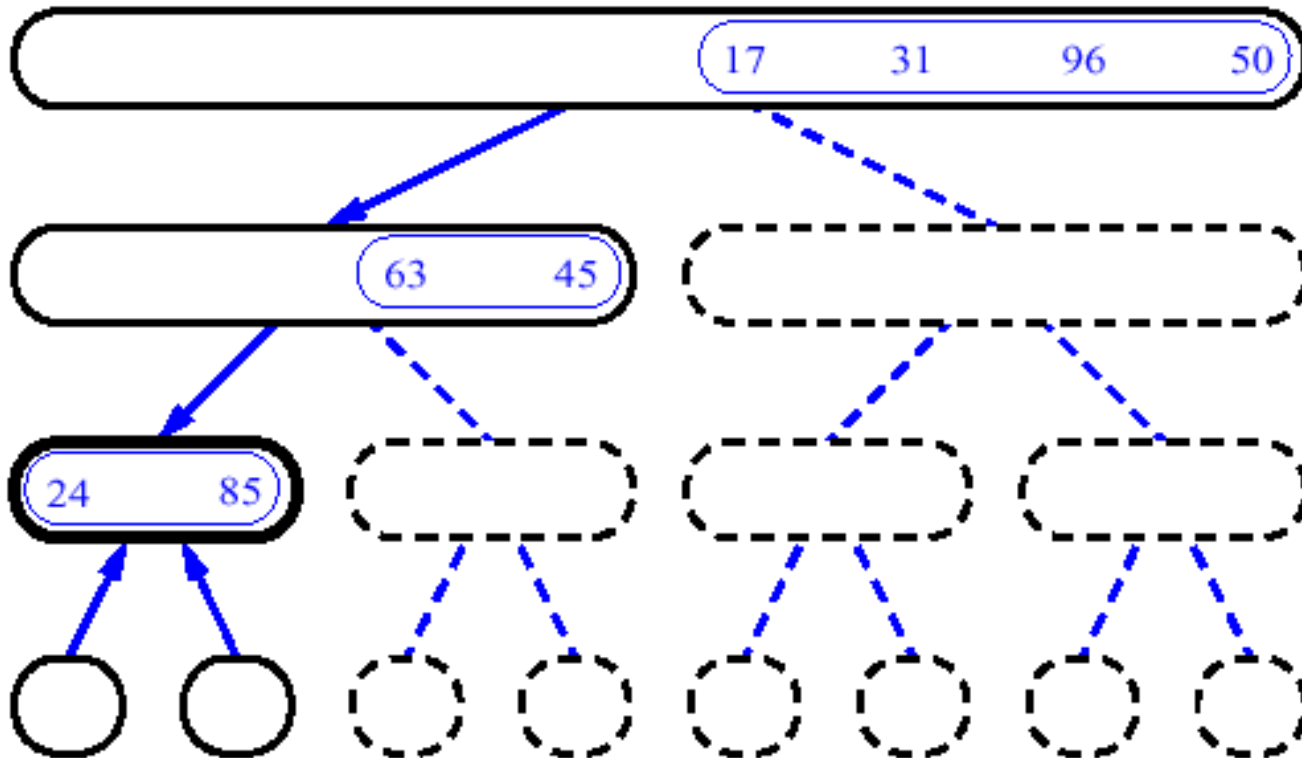
MergeSort (Example) - 6



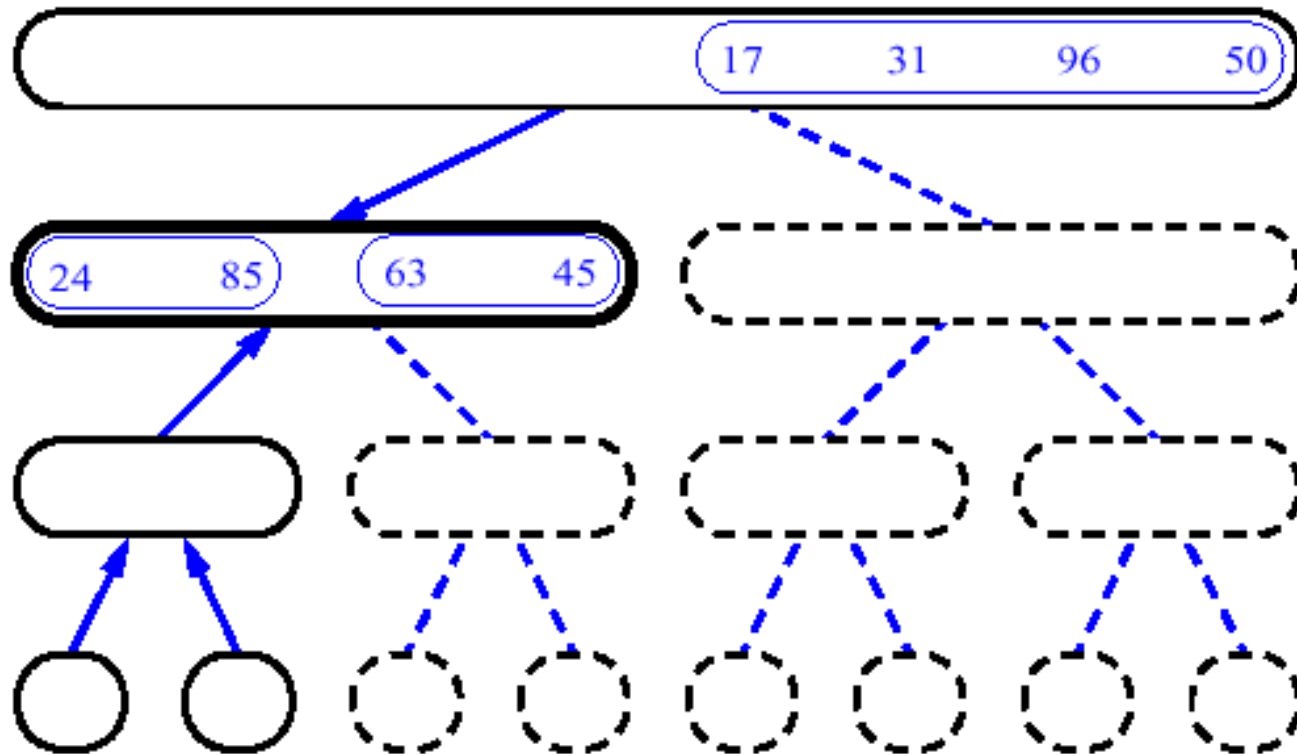
MergeSort (Example) - 7



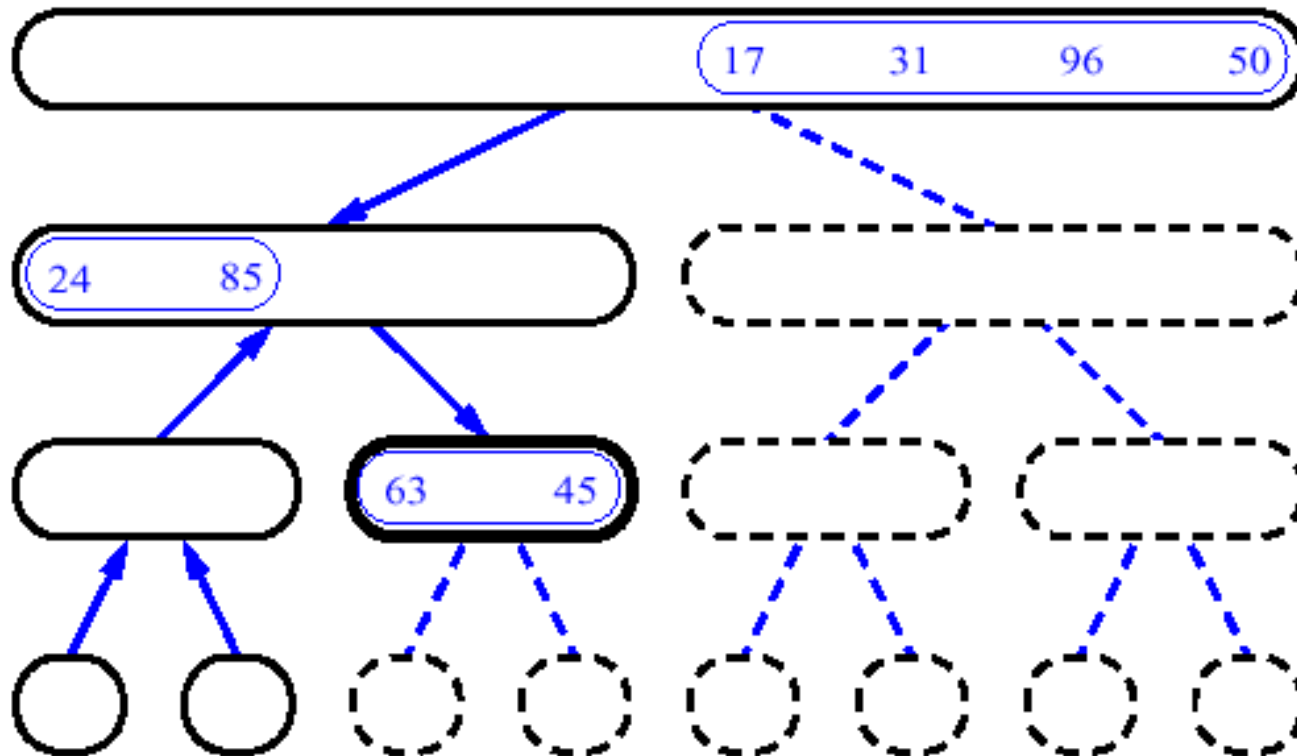
MergeSort (Example) - 8



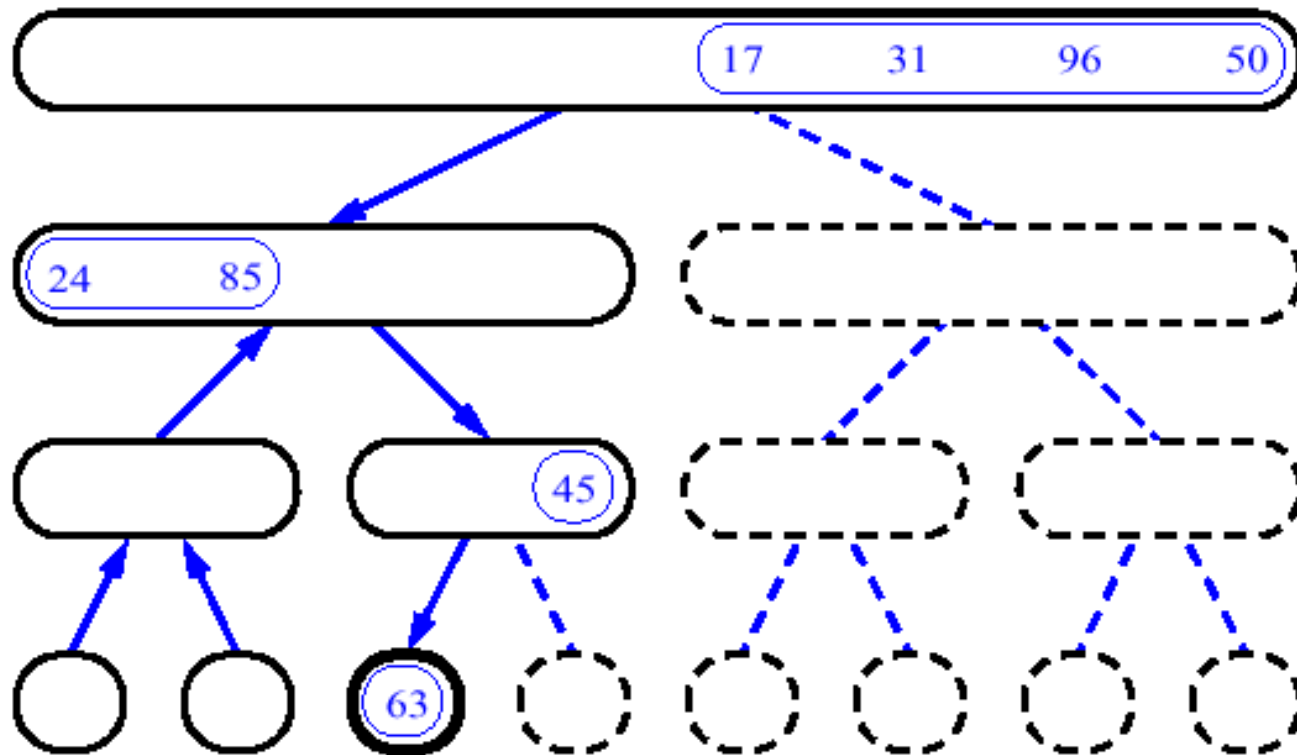
MergeSort (Example) - 9



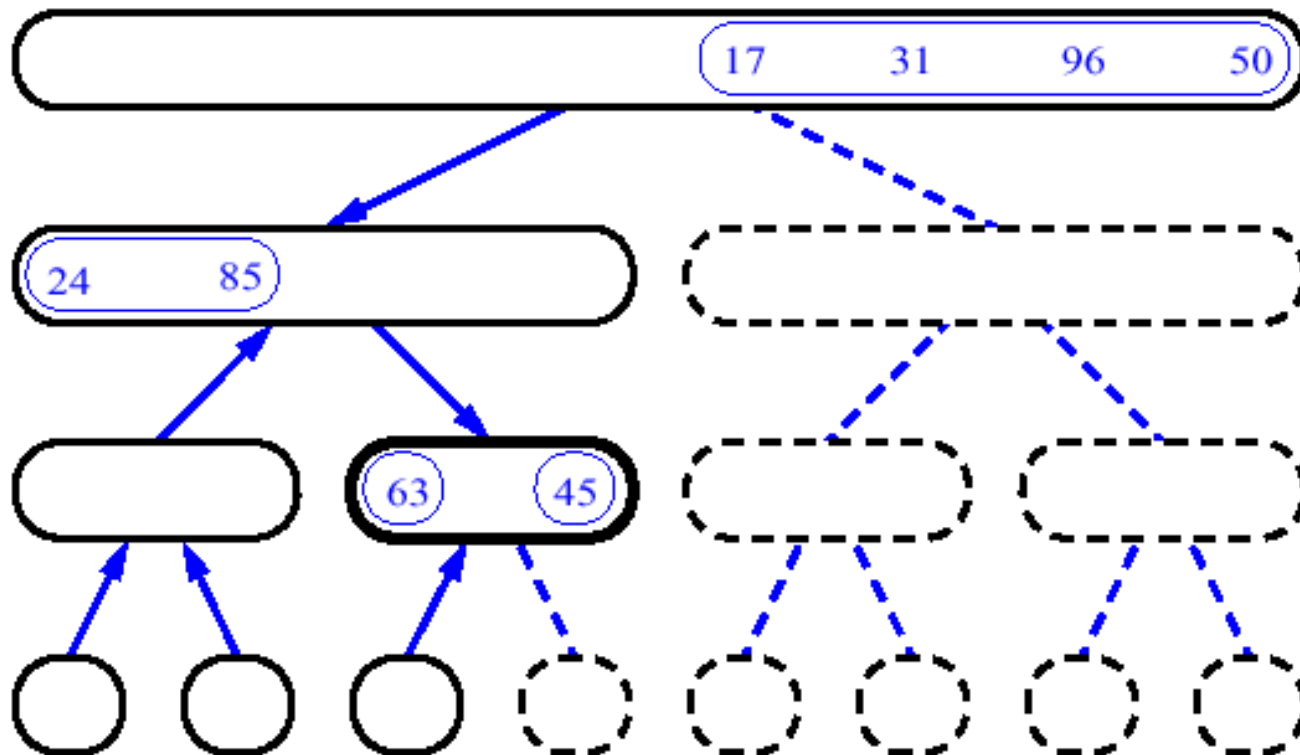
MergeSort (Example) - 10



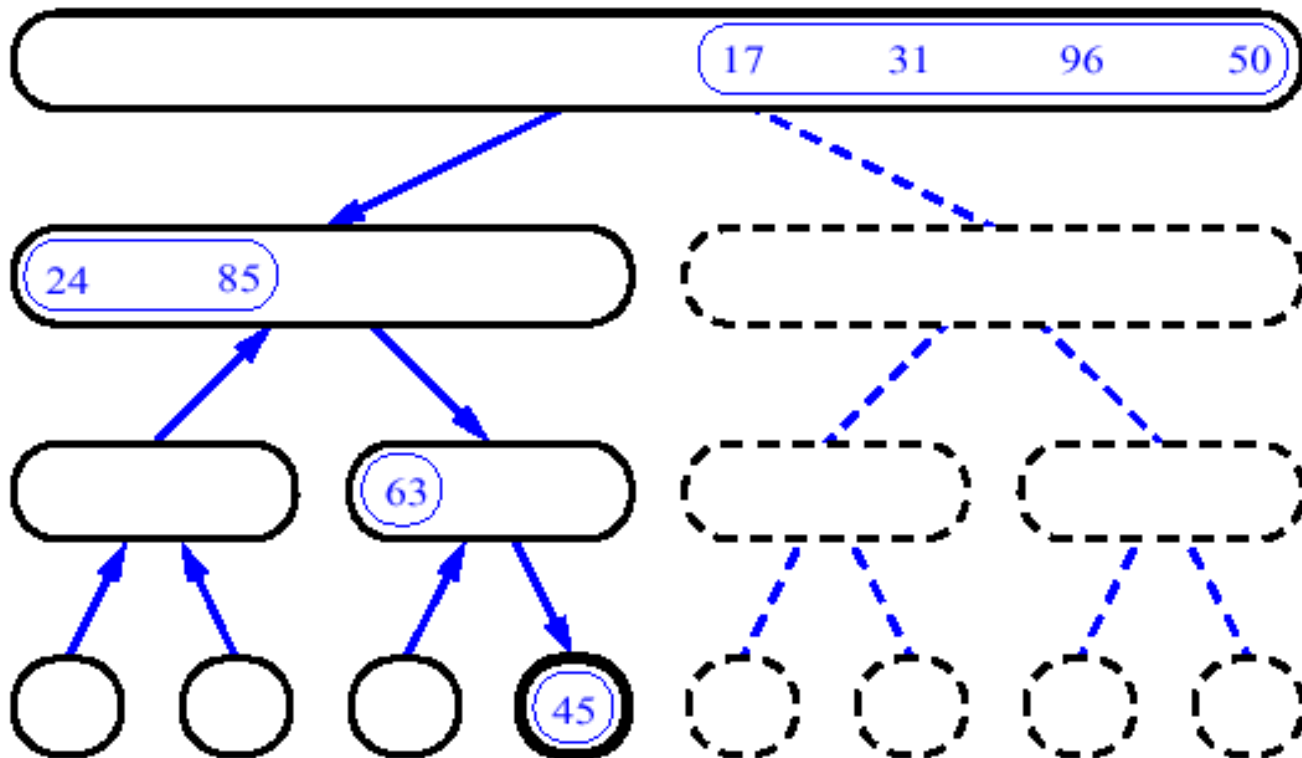
MergeSort (Example) - 11



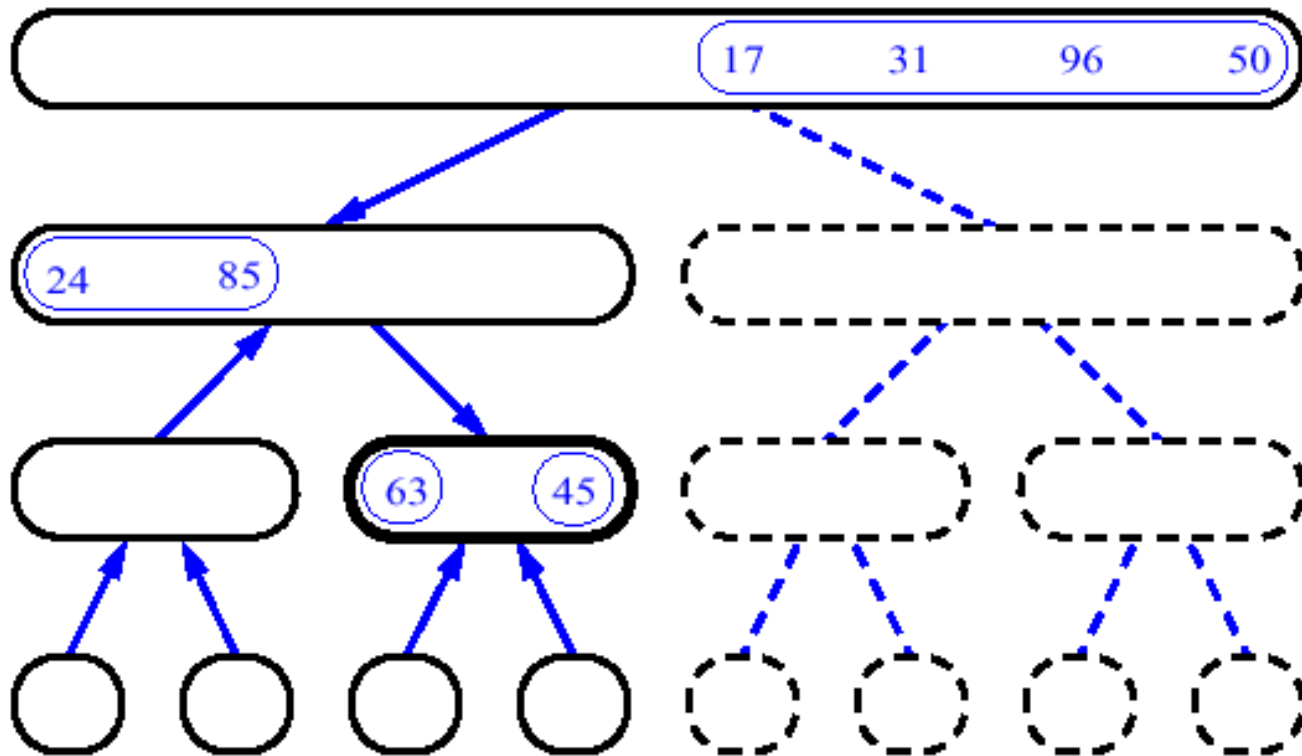
MergeSort (Example) - 12



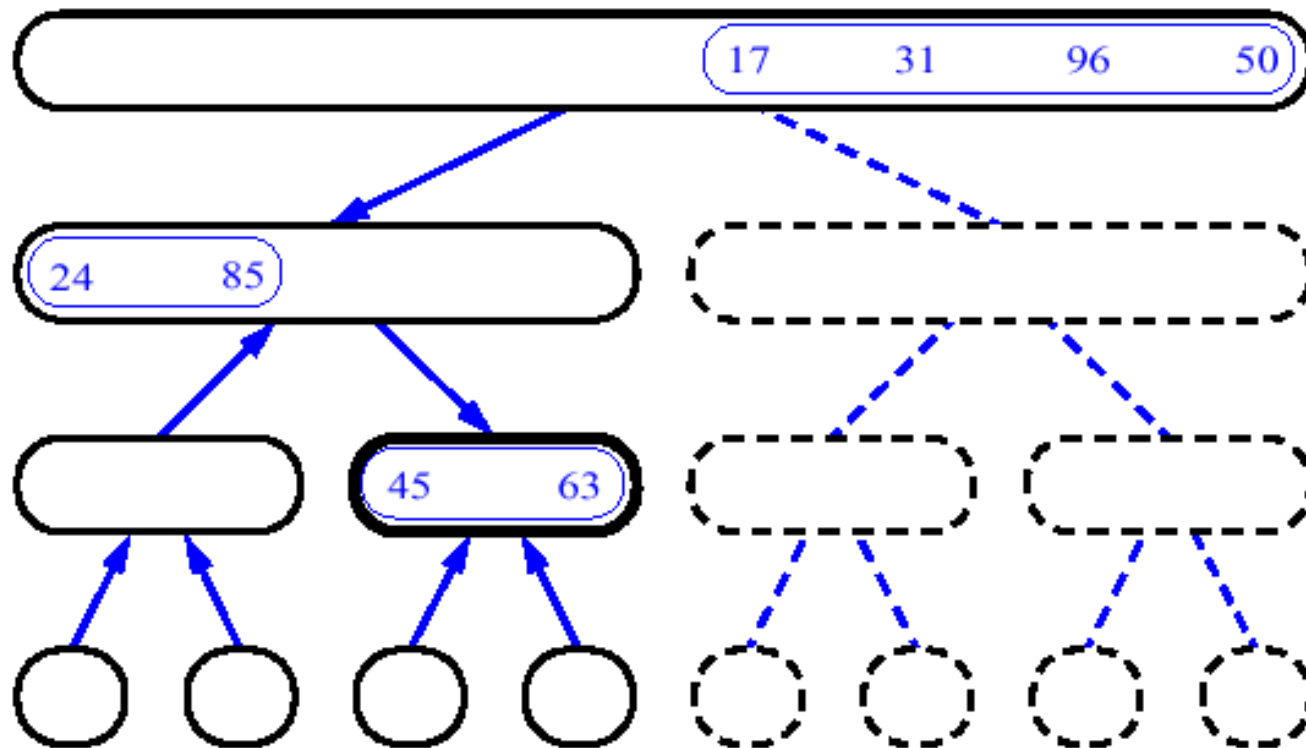
MergeSort (Example) - 13



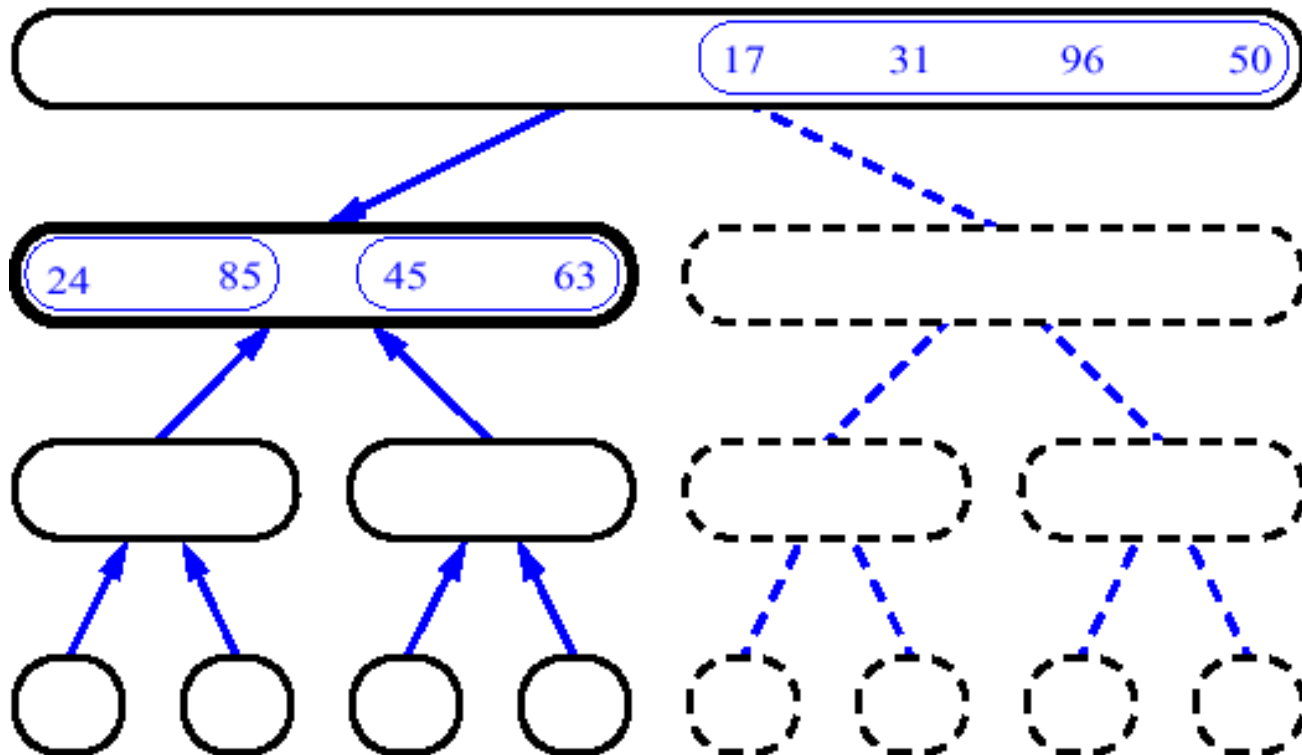
MergeSort (Example) - 14



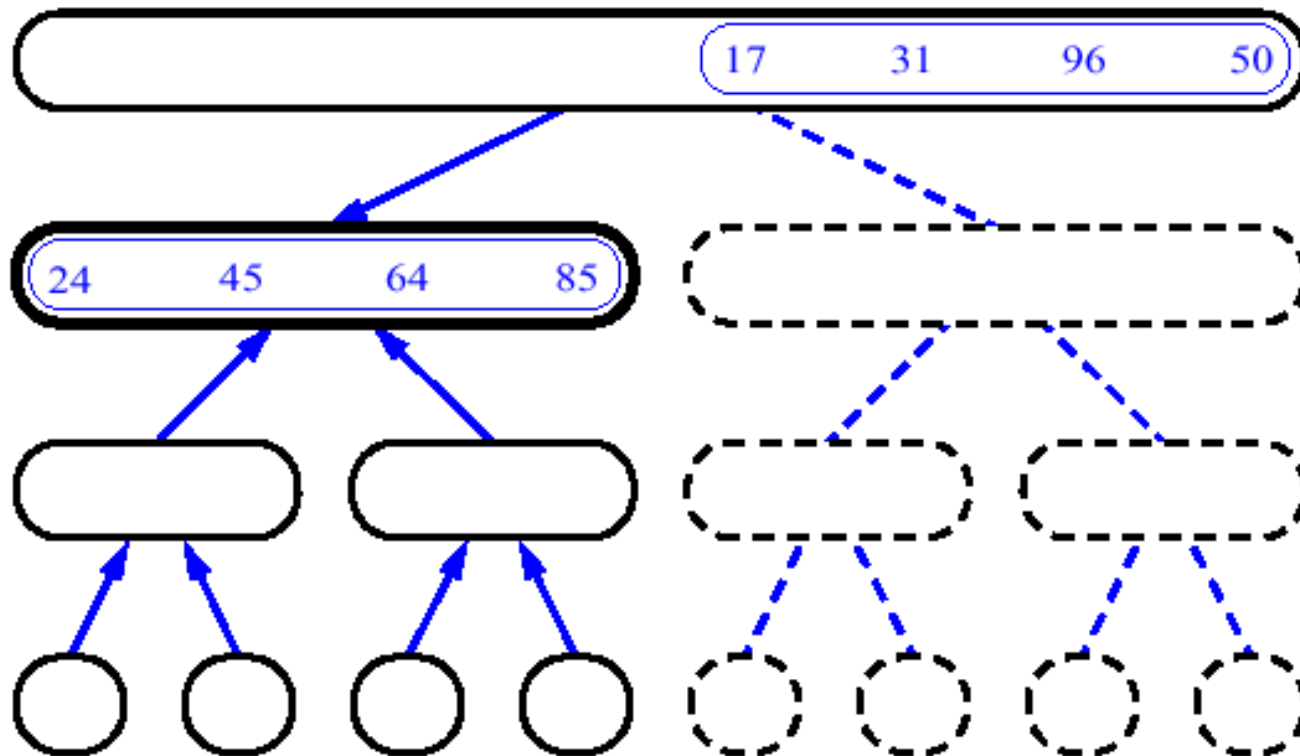
MergeSort (Example) - 15



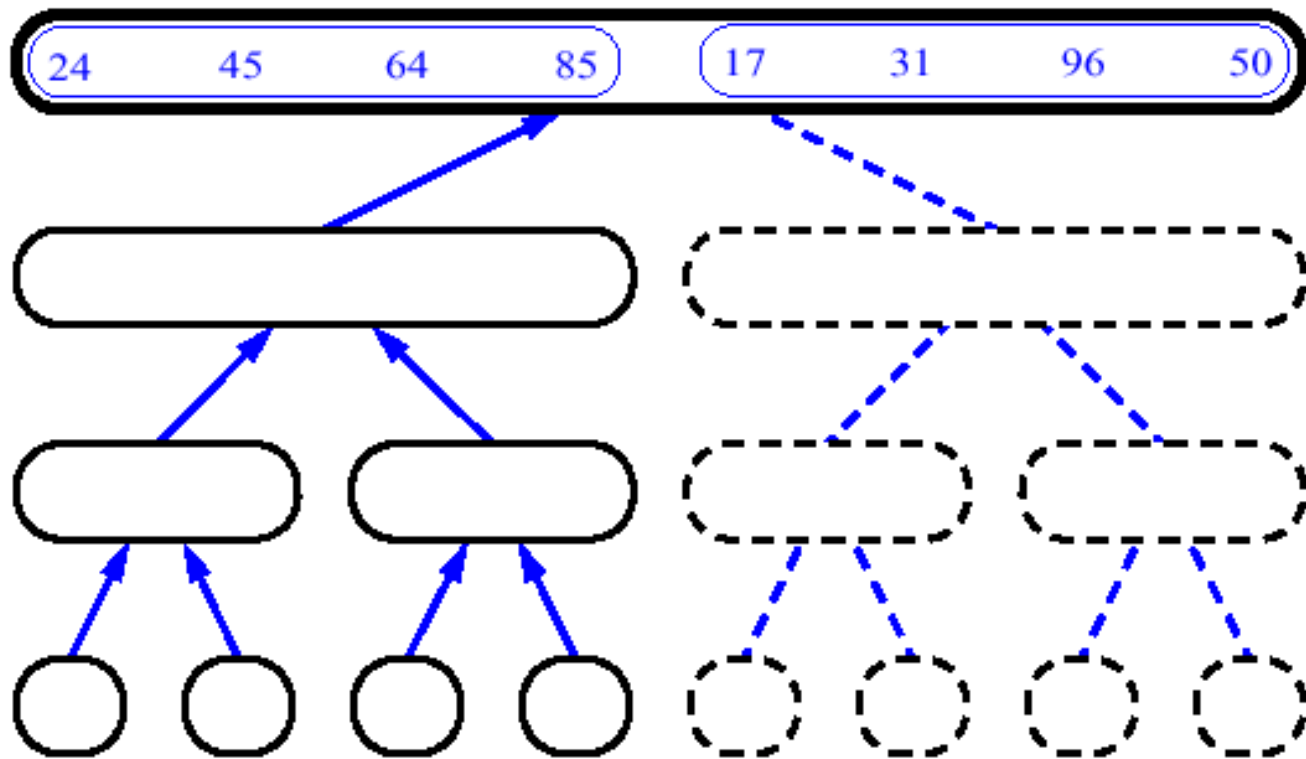
MergeSort (Example) - 16



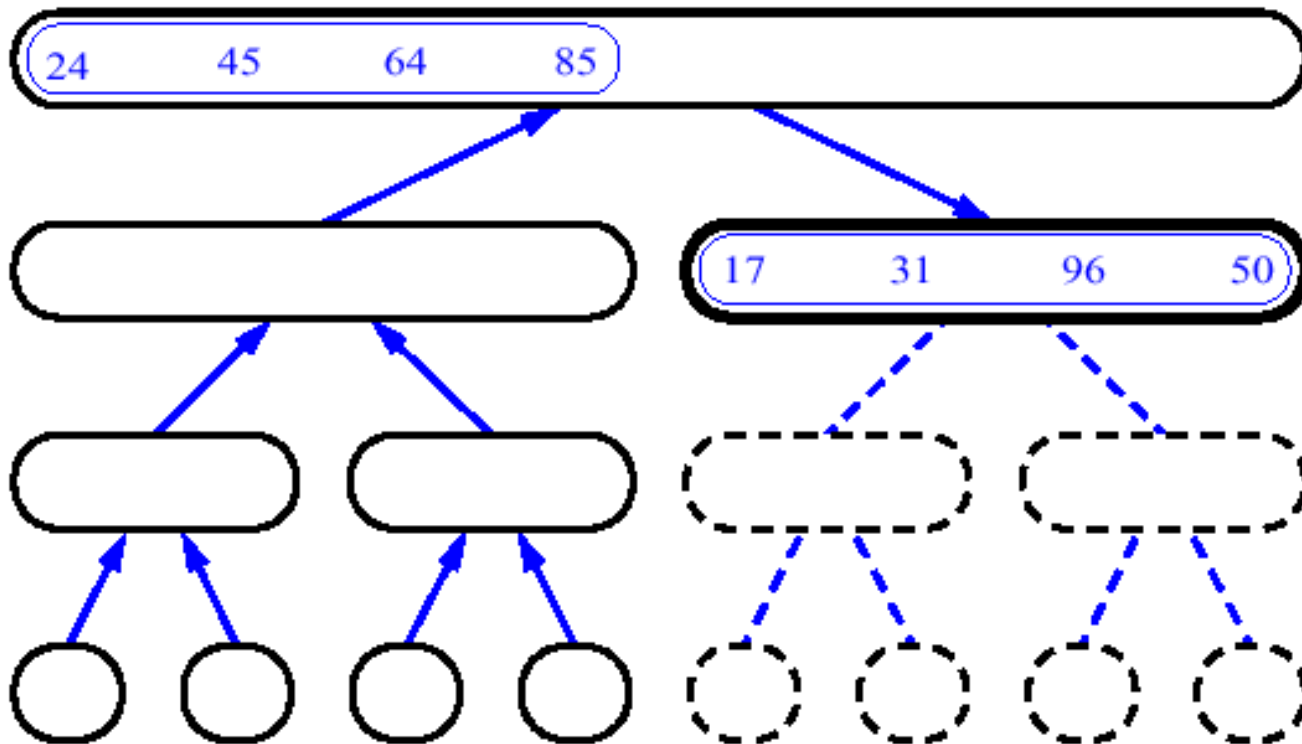
MergeSort (Example) - 17



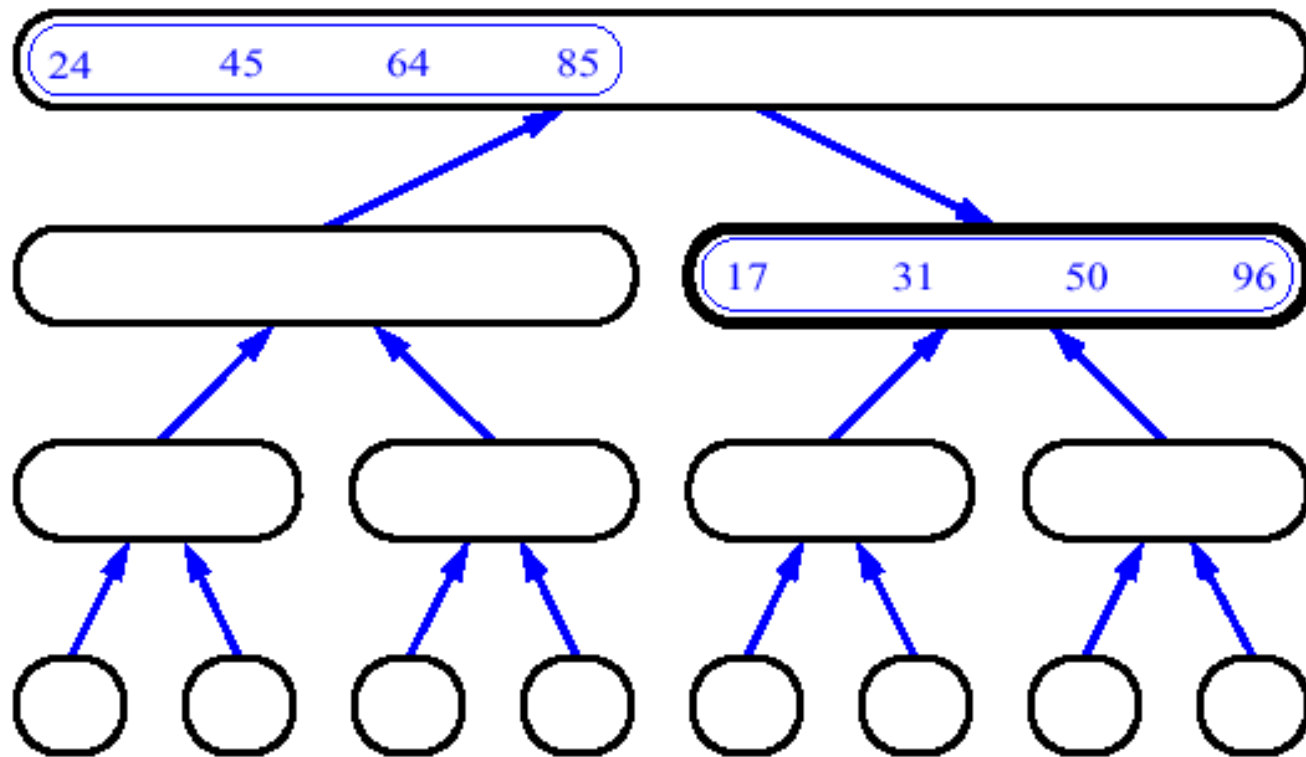
MergeSort (Example) - 18



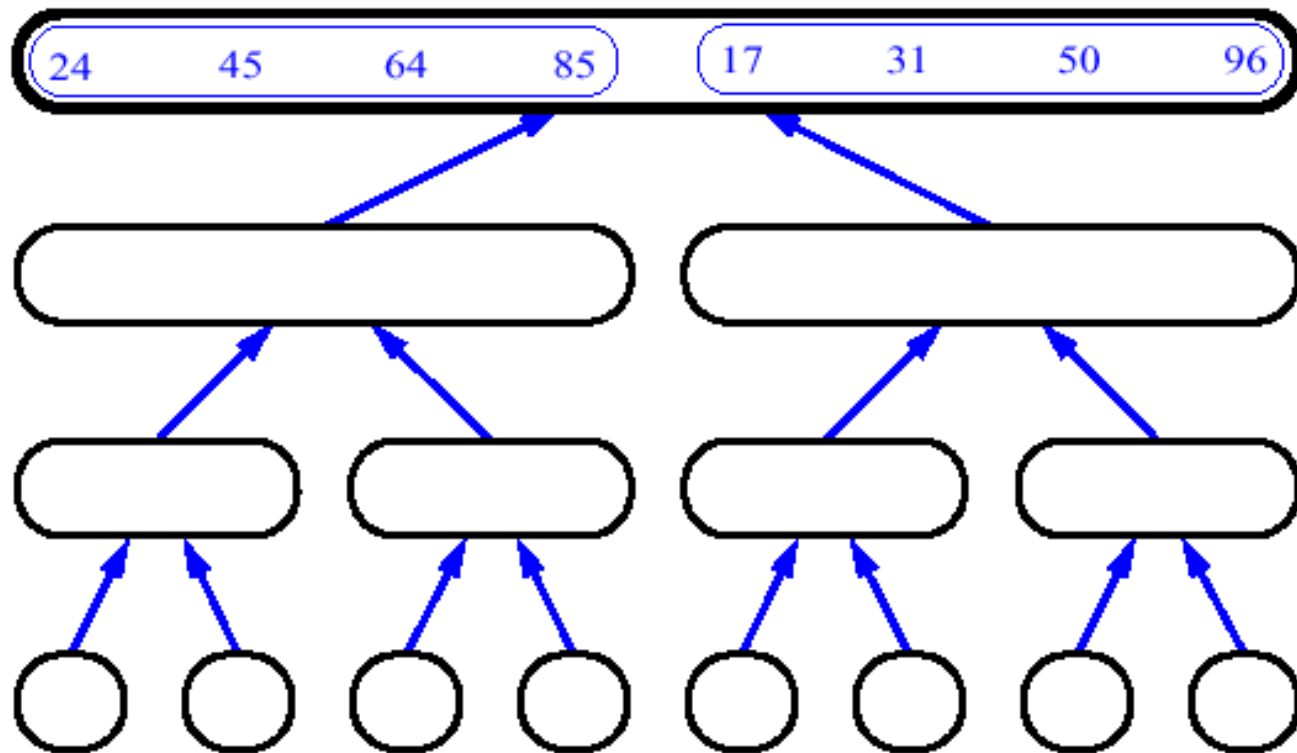
MergeSort (Example) - 19



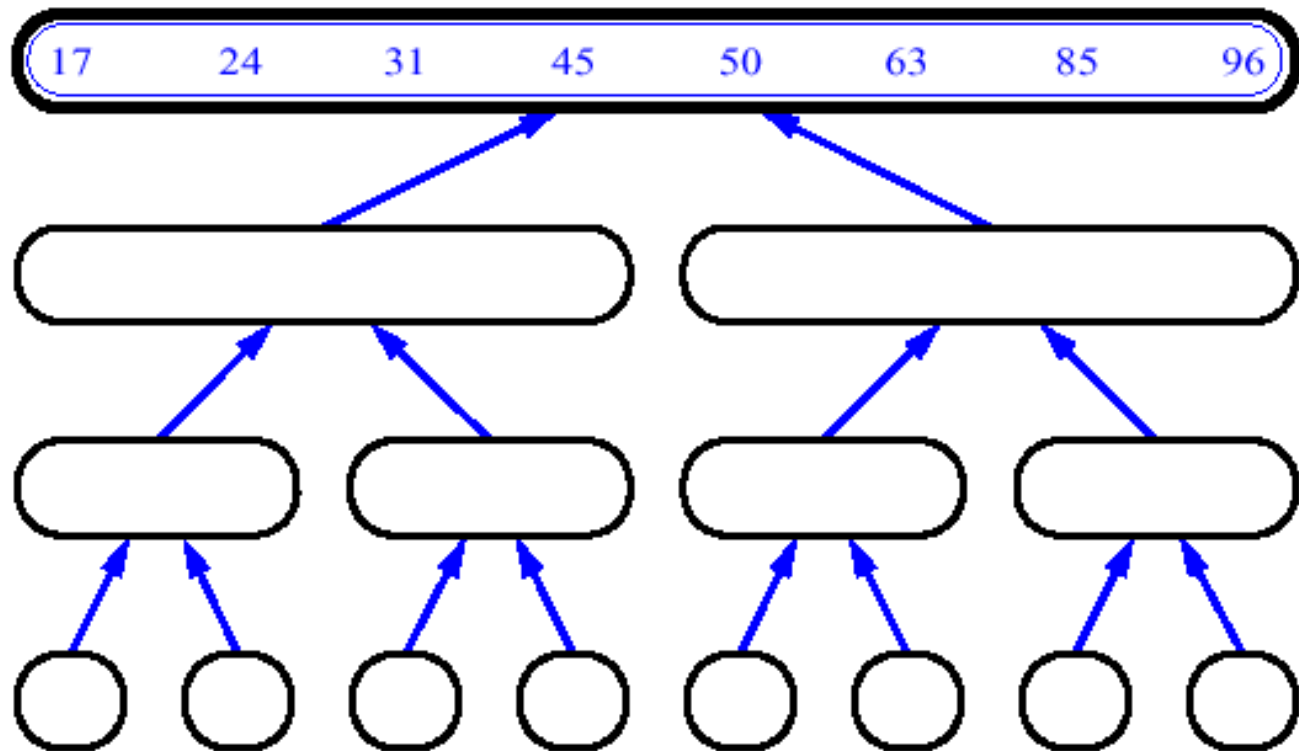
MergeSort (Example) - 20



MergeSort (Example) - 21



MergeSort (Example) - 22



Merge

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

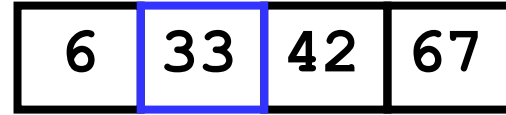
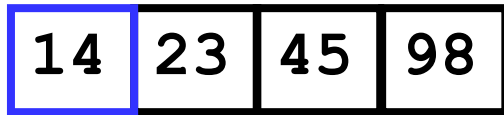
Merge

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

Merge

Merge



Merge

Merge

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14
---	----

Merge

Merge

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23
---	----	----

Merge

Merge

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33
---	----	----	----

Merge

Merge

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33	42
---	----	----	----	----

Merge

Merge

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33	42	45
---	----	----	----	----	----

Merge

Merge

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33	42	45	67
---	----	----	----	----	----	----

Merge

Merge

14	23	45	98
----	----	----	----

6	33	42	67
---	----	----	----

6	14	23	33	42	45	67	98
---	----	----	----	----	----	----	----

Merge

Merge Sort Algorithm

```
proc mergeSort(arr[1...n], p, r)
```

```
  if p < r then
```

```
    q ← (p+r)/2
```

```
    mergeSort(arr, p, q)
```

```
    mergeSort(arr, q+1, r)
```

```
    merge(arr, p, q, r)
```

```
  end if
```

```
end proc
```

- ✓ Every pass of `mergeSort()` procedure consists of two recursive function calls to itself followed by one call to `merge`
- ✓ `mergeSort(arr, p, r)` function checks whether `p` (index) is less than `r` (index)
- ✓ If it is so, it makes two calls to itself splitting the array into two halves
- ✓ Every further call splits array into further two halves with available indexes until `p` is not less than `r` (**when will this happen?**)
- ✓ Every further call splits array into further two halves with available indexes until `p` is not less than `r` (**when will this happen?**)
- ✓ At this point a `merge(arr, p, q, r)` call will be made to merge two halves into a sorted chunk

Merge Sort ... **merge(arr, p, q, r)**Call

```
proc merge(arr, p, q, r)
  i ← p, k ← p, j ← q + 1
  temp[p...r] // using a temporary array
  while i ≤ q and j ≤ r do
    if arr[i] ≤ arr[j] then
      temp[k++] ← arr[i++]
    else
      temp[k++] ← arr[j++]
    end if
  if i > q then
    While j ≤ r do
      temp[k++] = arr[j++]
    else
      While i ≤ q do
        temp[k++] = arr[i++]
      end if
    end proc
```

Quicksort

- Like merge sort, Quicksort is also based on the *divide-and-conquer* paradigm.
- Quicksort is a divide and conquer algorithm, which relies on a partition operation: to partition an array, we choose an element, called a pivot, move all smaller elements before the pivot, and move all greater elements after it.
- This can be done efficiently in linear time and in-place. We then recursively sort the lesser and greater sub lists. Efficient implementations of quicksort (with in-place partitioning) are somewhat complex, but are among the fastest sorting algorithms in practice.

Quicksort (cont.)

The most complex issue in quicksort is choosing a good pivot element; consistently poor choices of pivots can result in drastically slower.

The quick-sort algorithm consists of the following three steps:

1. ***Divide***: Partition the list.

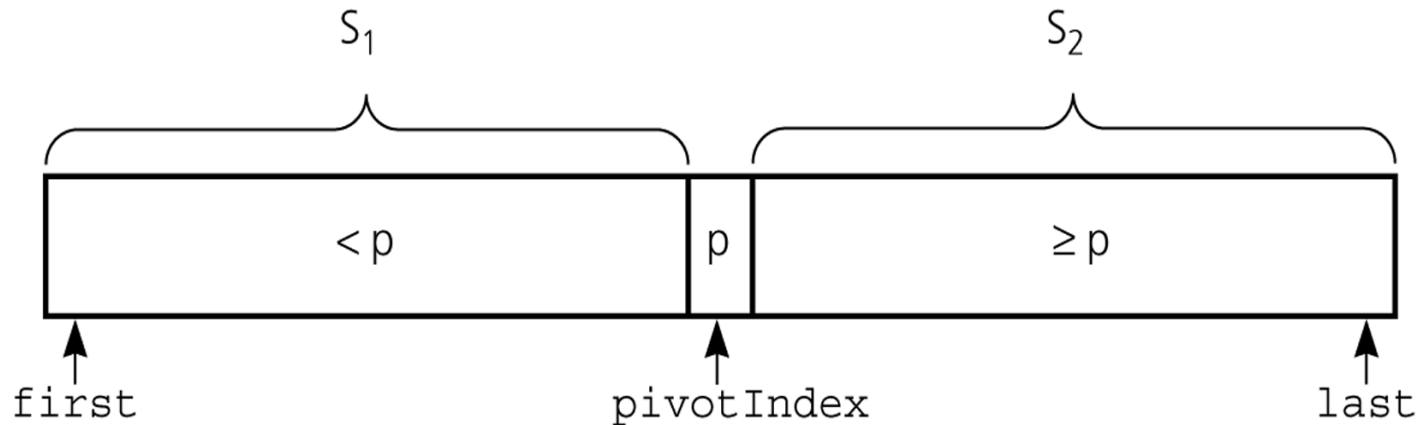
- To partition the list, we first choose some element from the list for which we hope about half the elements will come before and half after. Call this element the ***pivot***.
- Then we partition the elements so that all those with values less than the pivot come in one sub list and all those with greater values come in another.

2. ***Recursion***: Recursively sort the sub lists separately.

3. ***Conquer***: Put the sorted sub lists together.

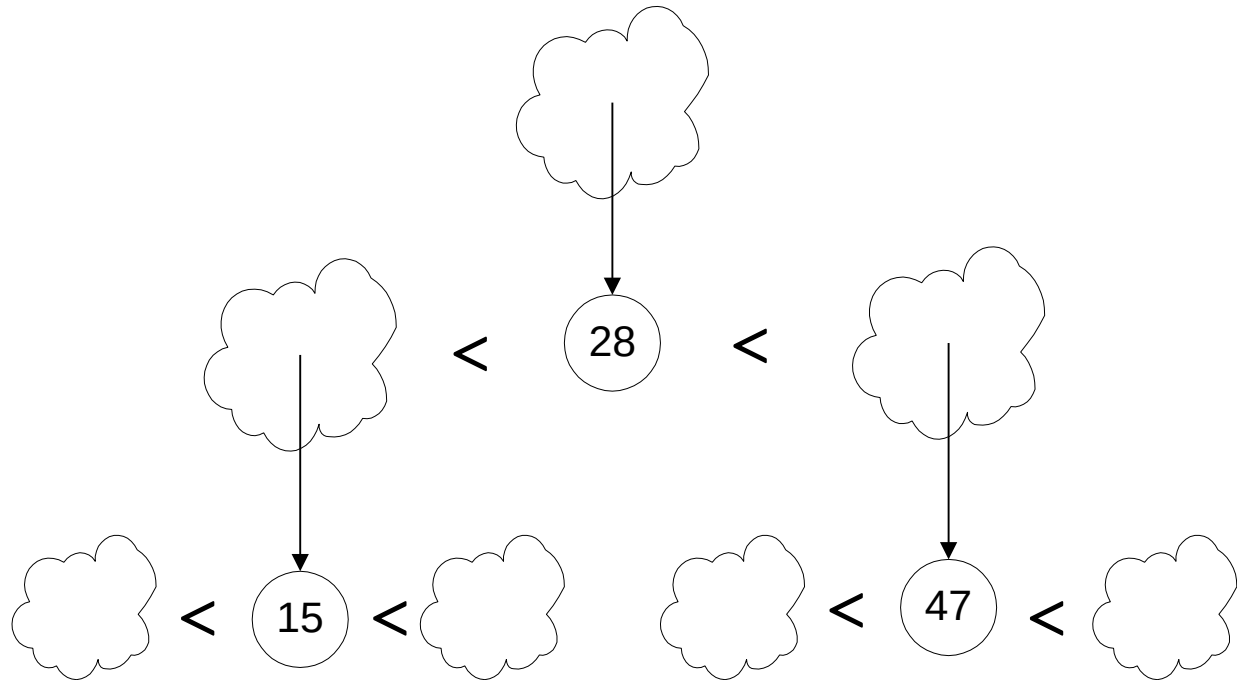
Partition

- Partitioning places the pivot in its correct place position within the array.



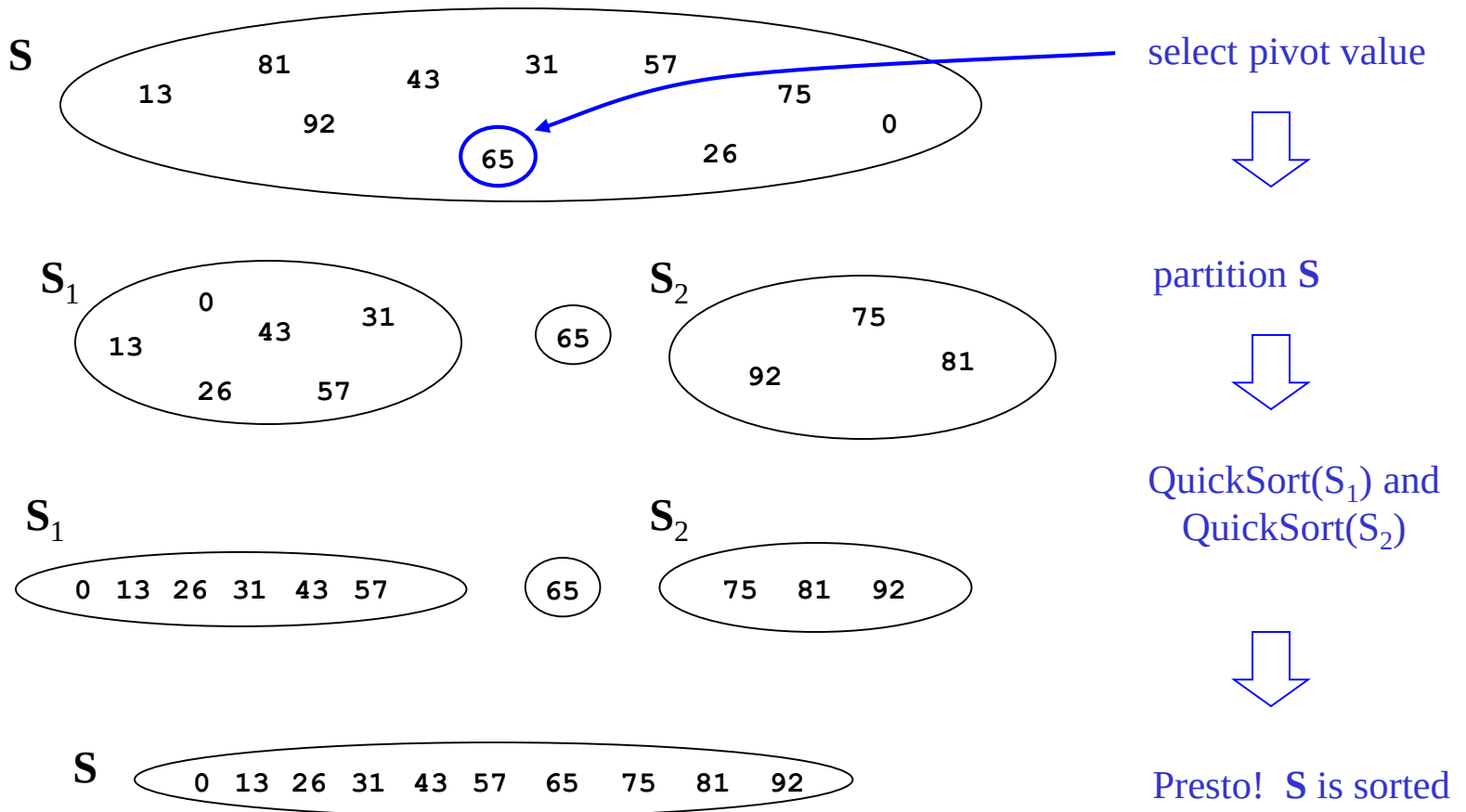
- Arranging the array elements around the pivot p generates two smaller sorting problems.
 - sort the left section of the array, and sort the right section of the array.
 - when these two smaller sorting problems are solved recursively, our bigger sorting problem is solved.

Quick Sort



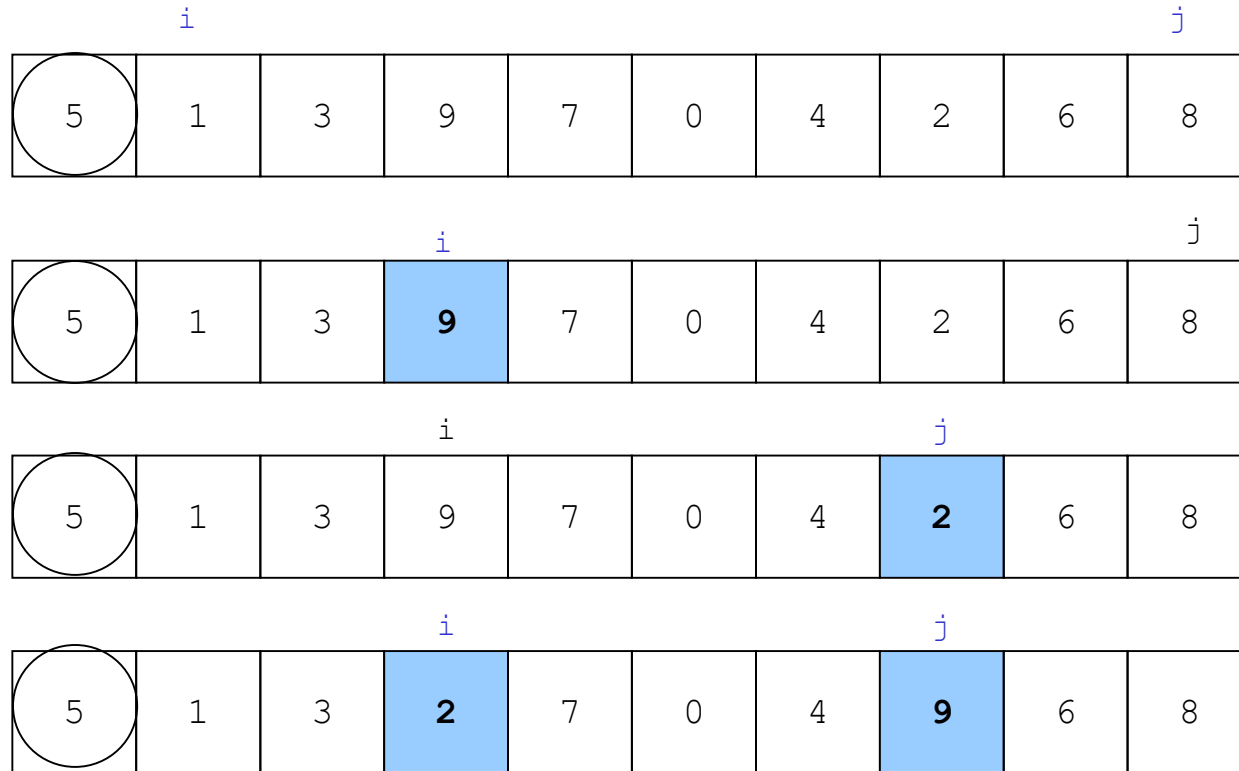
1. Pick a “pivot”
2. Divide into less-than & greater-than pivot
3. Sort each side recursively

The steps of QuickSort



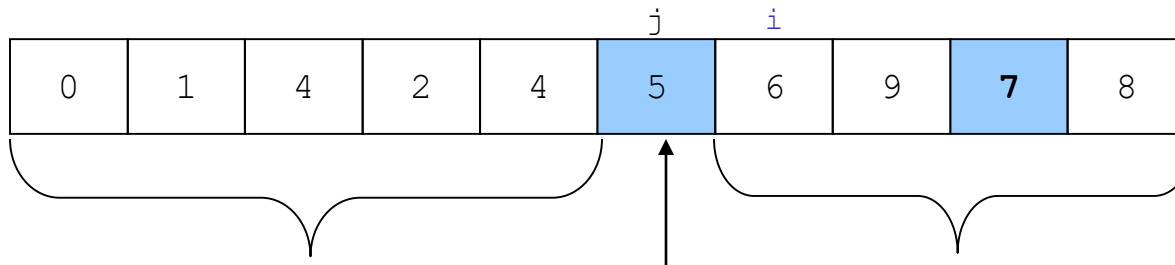
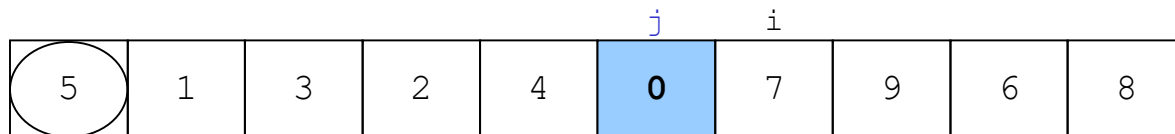
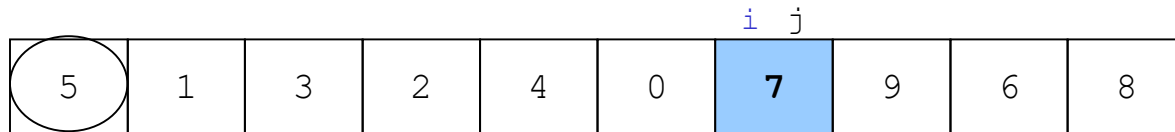
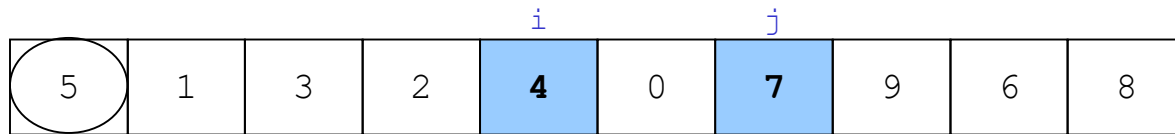
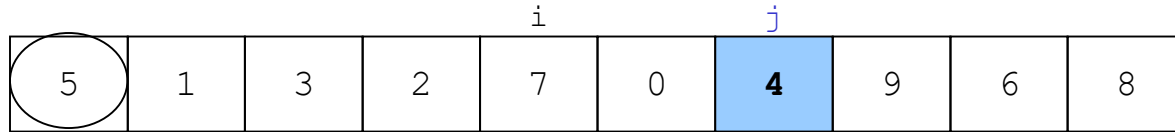
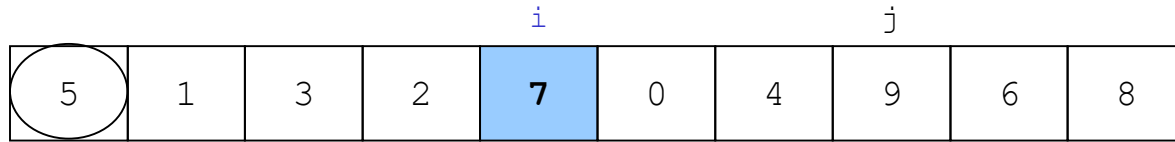
[Weiss]

QuickSort Example



- Move *i* to the right to be larger than pivot.
- Move *j* to the left to be smaller than pivot.
- Swap

QuickSort Example



$S_1 < \text{pivot}$

pivot

$S_2 > \text{pivot}$

Partition – Choosing the pivot

- First, we have to select a pivot element among the elements of the given array, and we put this pivot into the first location of the array before partitioning.
- Which array item should be selected as pivot?
 - Somehow we have to select a pivot, and we hope that we will get a good partitioning.
 - If the items in the array arranged randomly, we choose a pivot randomly.
 - We can choose the first or last element as a pivot (it may not give a good partitioning).
 - We can use different techniques to select the pivot.

Quick sort vs. Merge sort

Quick sort:

- pick a pivot value from the array
- partition the list around the pivot value
- sort the left half
- sort the right half

Merge sort:

- divide a list into two identically sized halves
- sort the left half
- sort the right half
- recombine the sorted halves into a sorted whole

Quicksort Function

```
QuickSort (A, lb, Ub)
{
    if (lb < Ub)
    {
        location = Partition (A, lb, Ub);
        QuickSort (A, lb, location - 1);
        QuickSort (A, location + 1 , Ub);
    }
}
```


Partition Function

```
Partition (A, lb, Ub)
{
    pivot = a[lb]; i = lb; j = Ub;
    while( i < j )
    {
        while( a[i] <= pivot )
            { i++; }
        while( a[j] > pivot )
            { j--; }
        if (i < j)
            { swap(a[i], a[j]); }
    }
    swap(a[lb], a[j]);
    return j;
}
```